

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH TRƯỜNG
ĐẠI HỌC BÁCH KHOA**

Khoa Khoa học và Kỹ thuật Máy tính



**BÀI TẬP LỚN
HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU**

**ĐỀ TÀI: SO SÁNH 2 DBMS REDIS VÀ MYSQL, TRIỂN KHAI HỆ THỐNG
HỖ TRỢ TUTOR CHO SINH VIÊN BÁCH KHOA Nhóm: 9**

GVHD: Cô Lê Thị Bảo Thu

STT	Họ và tên	MSSV
1	Lê Triều Kha	2311388
2	Trương Thanh Nhân	2312453
3	Nguyễn Hữu Thời	2313341
4	Huỳnh Hữu Nhật	2312469
5	Hà Thành Trung	2233137

BẢNG PHÂN VIỆC

MSSV	Họ tên	Công việc	HT
2311388	Lê Triều Kha	Thực hiện phần so sánh concurrency.	100%
2312453	Trương Thanh Nhân	Thực hiện phần so sánh indexing.	100%
2312469	Huỳnh Hữu Nhật	Thực hiện phần data storage, management.	100%
2313341	Nguyễn Hữu Thời	Thực hiện phần so sánh transaction.	100%
2233137	Hà Thành Trung	Thực hiện phần query processing.	100%

Bảng 1: Bảng phân việc nhóm

MỤC LỤC

1	Overview	5
1.1	Giới thiệu MySQL	5
1.2	Giới thiệu Redis	5
1.3	Mục tiêu của đề tài	6
1.4	Phạm vi nghiên cứu	6
2	So sánh 2 DBMS trên các phương diện	7
2.1	Data Storage and Management	7
2.1.1	Mục tiêu	7
2.1.2	Cơ chế lưu trữ vật lý (Physical Storage)	7
2.1.3	Quản lý bộ nhớ và Tối ưu hóa chuyên sâu	8
2.1.4	Cơ chế độ bền dữ liệu (Data Persistence)	8
2.1.5	Benchmark	9
2.1.6	Kết luận	16
2.2	Indexing	18
2.2.1	Mục tiêu	18
2.2.2	Các loại index trong MySQL và Redis	18
2.2.3	Benchmark	19
2.2.4	Tạo Sorted Set index trong Redis	28
2.2.5	Kết luận	29
2.3	Query Processing	31
2.3.1	Khái niệm Query Processing	31
2.3.2	Cách tiếp cận của MySQL và Redis	31
2.3.3	Môi trường và dữ liệu thử nghiệm	32
2.3.4	Thực nghiệm truy vấn	32
2.3.5	Phân tích sâu hơn về khả năng biểu diễn truy vấn	39
2.3.6	Tổng hợp ưu và nhược điểm	40
2.3.7	Kết luận	40
2.4	Transaction	41
2.4.1	Mô hình ACID trong MySQL và Redis	41
2.4.2	Hành vi Transaction trong các kịch bản thực tế	42
2.4.3	Nhận xét so sánh	53
2.5	Concurrency	54
2.5.1	Khái niệm Concurrency Control	54

2.5.2	Kiến trúc 2 DBMS	54
2.5.3	Thực nghiệm	55
2.5.4	Cơ chế lock trong MySQL	57
2.5.5	Cơ chế MVCC của MySQL	61
2.5.6	Deadlock trong MySQL	62
2.5.7	Tính nguyên tử của Redis	63
2.5.8	Kết luận	63

CHƯƠNG 1

OVERVIEW

1.1 Giới thiệu MySQL

MySQL là một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS - Relational Database Management System) mã nguồn mở phổ biến nhất trên thế giới. Được phát triển bởi MySQL AB vào năm 1995 và hiện thuộc sở hữu của Oracle Corporation, MySQL đã trở thành lựa chọn hàng đầu cho các ứng dụng web và doanh nghiệp trên toàn cầu.

Đặc điểm nổi bật của MySQL:

- **Mô hình dữ liệu quan hệ:** MySQL sử dụng mô hình dữ liệu quan hệ với các bảng (tables), hàng (rows), và cột (columns). Dữ liệu được tổ chức theo cấu trúc chặt chẽ với schema được định nghĩa trước.
- **Ngôn ngữ SQL:** Hỗ trợ đầy đủ ngôn ngữ truy vấn có cấu trúc SQL (Structured Query Language), cho phép thực hiện các thao tác CRUD (Create, Read, Update, Delete) và các truy vấn phức tạp.
- **Hỗ trợ ACID:** Đảm bảo tính Atomicity (nguyên tử), Consistency (nhất quán), Isolation (cô lập), và Durability (bền vững) cho các giao dịch.
- **Hỗ trợ nhiều Storage Engine:** Bao gồm InnoDB (mặc định), MyISAM, Memory, và nhiều engine khác, mỗi loại tối ưu cho các use case khác nhau.
- **Khả năng mở rộng:** Hỗ trợ replication (master-slave, master-master), clustering, và partitioning để mở rộng quy mô hệ thống.

1.2 Giới thiệu Redis

Redis (Remote Dictionary Server) là một hệ thống lưu trữ dữ liệu dạng key-value trong bộ nhớ (in-memory), mã nguồn mở, được phát triển bởi Salvatore Sanfilippo vào năm 2009. Redis được thiết kế để đạt hiệu suất cực cao với khả năng xử lý hàng triệu thao tác mỗi giây.

Đặc điểm nổi bật của Redis:

- **Lưu trữ trong bộ nhớ (In-Memory):** Redis lưu trữ toàn bộ dữ liệu trong RAM, cho phép truy cập với độ trễ cực thấp (microseconds).
- **Cấu trúc dữ liệu đa dạng:** Hỗ trợ nhiều kiểu dữ liệu như Strings, Hashes, Lists, Sets, Sorted Sets, Bitmaps, HyperLogLogs, và Streams.

- **Persistence (Bền vững):** Hỗ trợ RDB (Redis Database Backup) snapshots và AOF (Append Only File) để đảm bảo dữ liệu không bị mất khi hệ thống khởi động lại.
- **Pub/Sub Messaging:** Cung cấp cơ chế publish/subscribe cho việc giao tiếp giữa các ứng dụng theo thời gian thực.
- **Transactions và Lua Scripting:** Hỗ trợ giao dịch với lệnh MULTI/EXEC và khả năng thực thi script Lua phía server.
- **High Availability:** Redis Sentinel cung cấp khả năng giám sát, thông báo, và tự động failover. Redis Cluster hỗ trợ sharding dữ liệu tự động.

1.3 Mục tiêu của đề tài

Đề tài này được thực hiện với các mục tiêu chính sau:

1. **Nghiên cứu và so sánh hai hệ quản trị cơ sở dữ liệu:** Phân tích chuyên sâu về kiến trúc, tính năng, và nguyên lý hoạt động của MySQL (đại diện cho RDBMS truyền thống) và Redis (đại diện cho NoSQL in-memory database).
2. **So sánh hiệu năng:** Đánh giá và so sánh hiệu suất của hai hệ thống trên các tiêu chí:
 - Tốc độ đọc/ghi dữ liệu
 - Khả năng xử lý đồng thời (concurrency)
 - Độ trễ (latency) và throughput
 - Sử dụng tài nguyên hệ thống (CPU, RAM, Disk I/O)
3. **Phân tích cơ chế kiểm soát đồng thời:** Nghiên cứu và so sánh các cơ chế:
 - Các mức độ isolation trong MySQL (READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE)
 - Cơ chế xử lý đồng thời single-threaded của Redis
 - Lock mechanisms và MVCC (Multi-Version Concurrency Control)
4. **Xây dựng bộ test benchmark:** Thiết kế và triển khai các test case để đo lường và so sánh hiệu năng thực tế của hai hệ thống trong các tình huống khác nhau.
5. **Đề xuất use case phù hợp:** Dựa trên kết quả nghiên cứu, đưa ra các khuyến nghị về việc lựa chọn MySQL hoặc Redis cho các bài toán cụ thể trong thực tế.

1.4 Phạm vi nghiên cứu

Trong khuôn khổ đề tài này, nhóm tập trung nghiên cứu các khía cạnh sau:

- **MySQL:** Sử dụng phiên bản MySQL 8.0 với storage engine InnoDB (mặc định).
- **Redis:** Sử dụng phiên bản Redis 7.x với cả hai chế độ persistence (RDB và AOF).
- **Môi trường thử nghiệm:** Các thử nghiệm được thực hiện trên cùng một cấu hình phần cứng để đảm bảo tính công bằng khi so sánh.
- **Bộ dữ liệu:** Sử dụng bộ dữ liệu BikeStores và Sales Transaction để thực hiện các thử nghiệm.

CHƯƠNG 2

SO SÁNH 2 DBMS TRÊN CÁC PHƯƠNG DIỆN

2.1 Data Storage and Management

2.1.1 Mục tiêu

Mục tiêu của phần này là phân tích sự khác biệt cốt lõi trong kiến trúc lưu trữ vật lý, cách quản lý không gian và cơ chế độ bền dữ liệu giữa **MySQL** (Relational DBMS) và **Redis** (In-memory NoSQL). Báo cáo sẽ đi từ cấu trúc lưu trữ cơ bản đến các kỹ thuật tối ưu hóa chuyên sâu (như Eviction, Internal Encodings, Fragmentation), đồng thời thực hiện Benchmark để so sánh mức tiêu thụ tài nguyên lưu trữ (Storage Footprint) thông qua tập dữ liệu *BikeStores* và *Sales Transactions*.

2.1.2 Cơ chế lưu trữ vật lý (Physical Storage)

Lưu trữ trong MySQL (Disk-Based Storage) Trong MySQL, với *Storage Engine* mặc định là **InnoDB**, dữ liệu được thiết kế để lưu trữ vĩnh viễn trên ổ đĩa vật lý (HDD/SSD).

Cấu trúc Tablespace và Pages: Toàn bộ dữ liệu bảng và chỉ mục được lưu trong các tệp `.ibd`. Không gian này được chia nhỏ thành các trang dữ liệu (*Pages*) có kích thước cố định (mặc định 16KB).

Bộ nhớ đệm (Buffer Pool): Để giảm thiểu độ trễ vật lý (*Disk I/O*), InnoDB nạp các *Pages* thường xuyên truy cập từ đĩa lên một vùng RAM gọi là *Buffer Pool*. Khi có truy vấn, hệ thống ưu tiên đọc từ bộ nhớ đệm trước, nếu không tìm thấy mới thực hiện lệnh đọc đĩa.

Lưu trữ trong Redis (In-Memory Storage) Trái ngược với MySQL, Redis hoạt động hoàn toàn trên bộ nhớ chính (*RAM*) thông qua một *Global Hash Table*.

Truy xuất siêu tốc và Giới hạn dung lượng: Vì không có khái niệm *Page* hay *Disk I/O*, mỗi *Key* trong Redis trỏ trực tiếp đến địa chỉ ô nhớ chứa *Value*. Tốc độ đọc/ghi đạt mức micro-giây. Tuy nhiên, giới hạn lưu trữ phụ thuộc hoàn toàn vào dung lượng RAM hiện có của hệ thống máy chủ, khiến chi phí mở rộng lớn hơn nhiều so với việc mua ổ cứng lưu trữ.

2.1.3 Quản lý bộ nhớ và Tối ưu hóa chuyên sâu

Quản lý giới hạn không gian (Eviction Policies) Khi phân vùng lưu trữ bị đầy, hai hệ thống có cách xử lý hoàn toàn khác biệt:

- **MySQL:** Sẽ báo lỗi "*Disk Full*" và từ chối các lệnh `INSERT/UPDATE` mới. Quản trị viên phải can thiệp thủ công (thêm dung lượng đĩa hoặc xóa dữ liệu).
- **Redis:** Cho phép thiết lập cấu hình `maxmemory`. Khi RAM đầy, Redis tự động kích hoạt các chiến lược đào thải (*Eviction*) như **LRU** (Xóa Key lâu không dùng), **LFU** (Xóa Key ít dùng nhất), hoặc dựa trên **TTL** (Thời gian sống của Key). Cơ chế này giúp Redis tự duy trì hoạt động như một hệ thống *Cache* hoàn hảo.

Tối ưu hóa cấu trúc nội bộ (Internal Memory Encodings) Nhìn từ bên ngoài, lưu trữ *Hash* hay *Set* trong Redis tốn khá nhiều metadata (con trỏ bộ nhớ). Tuy nhiên, Redis tự động áp dụng các thuật toán nén tinh vi:

Cơ chế Ziplist/Listpack: Với các cấu trúc *Hash* có số lượng trường (*fields*) ít và chuỗi ngắn (như bản ghi sản phẩm trong `production.products`), Redis không khởi tạo *Hash Table* thực sự. Nó mã hóa toàn bộ dữ liệu thành một khối byte liên tục (*Ziplist*), loại bỏ hoàn toàn các con trỏ dư thừa để tiết kiệm tối đa dung lượng RAM.

Phân mảnh lưu trữ (Storage Fragmentation) Sau quá trình `INSERT`, `UPDATE`, `DELETE` liên tục, cả hai hệ thống đều sinh ra các khoảng trống vật lý vô ích.

- **MySQL:** Xảy ra phân mảnh đĩa (*Disk Fragmentation*) do hiện tượng *Page Split*. Quản trị viên cần định kỳ chạy lệnh `OPTIMIZE TABLE` để dọn đĩa.
- **Redis:** Xảy ra phân mảnh RAM do trình cấp phát bộ nhớ của OS (`jemalloc`) không thể tận dụng các mảnh RAM lẻ tẻ. Redis cung cấp tính năng *Active Defragmentation* giúp tự động dọn RAM ngầm dưới nền mà không làm gián đoạn hệ thống.

2.1.4 Cơ chế độ bền dữ liệu (Data Persistence)

MySQL: Đảm bảo ACID với Write-Ahead Logging Mọi thao tác thay đổi dữ liệu không được ghi trực tiếp xuống file `.ibd` ngay lập tức mà được ghi tuần tự vào **Redo Log** trên đĩa. Cơ chế này đảm bảo tính bền vững (*Durability*) tuyệt đối, giúp MySQL có thể khôi phục lại dữ liệu bị mất trên *Buffer Pool* nếu máy chủ đột ngột sập nguồn.

Redis: Tùy chọn RDB và AOF Do dữ liệu lưu trên RAM sẽ biến mất khi mất điện, Redis cung cấp hai giải pháp để sao lưu xuống đĩa cứng:

- **RDB (Redis Database Snapshot):** Định kỳ nén toàn bộ RAM thành một file nhị phân (`dump.rdb`). Khởi động lại nhanh, nhưng có thể mất dữ liệu ở giữa 2 lần snapshot.
- **AOF (Append Only File):** Ghi lại toàn bộ các lệnh thay đổi dữ liệu (`SET`, `HSET`) vào file log. An toàn hơn RDB nhưng làm file log phình to và thời gian phục hồi chậm hơn.

2.1.5 Benchmark

Test Case 1: Đánh giá không gian lưu trữ (Storage Footprint)

Mô tả kịch bản kiểm thử: Đánh giá và so sánh mức độ tiêu thụ không gian lưu trữ (trên đĩa vật lý và trên bộ nhớ RAM) khi hệ thống quản lý cùng một tập dữ liệu.

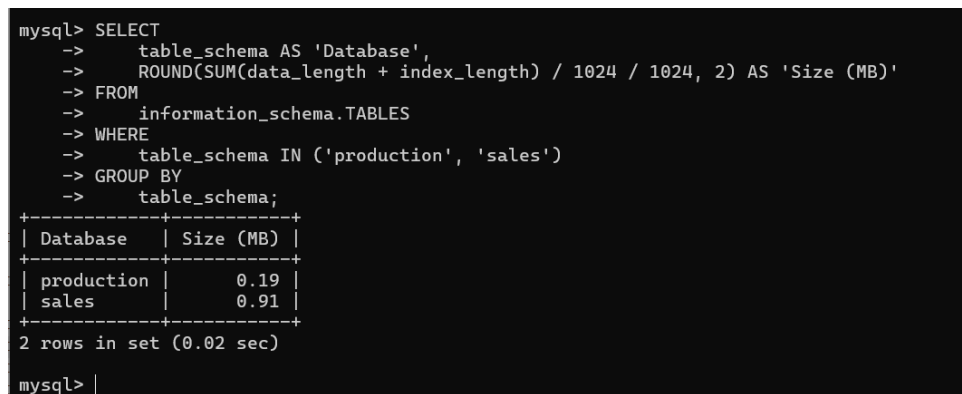
- **Dữ liệu sử dụng:** Toàn bộ cơ sở dữ liệu mẫu *BikeStores* (bao gồm 9 bảng phân bổ trong schema *production* và *sales*).
- **Yêu cầu:** Trích xuất tổng dung lượng lưu trữ (bao gồm Data và Index đối với MySQL) và dung lượng bộ nhớ cấp phát thực tế (đối với Redis) để đối chiếu.

1. Thực thi và đo lường trên MySQL (Disk Storage)

Phương pháp: Sử dụng bảng hệ thống `information_schema.TABLES` để tính tổng dung lượng vật lý bao gồm cả phần dữ liệu thô (`data_length`) và phần chỉ mục (`index_length`) của hai lược đồ *production* và *sales*.

```
SELECT table_schema AS 'Database',
       ROUND(SUM(data_length + index_length) / 1024 / 1024, 2) AS 'Size (MB)'
FROM   information_schema.TABLES
WHERE  table_schema IN ('production', 'sales')
GROUP BY table_schema;
```

Kết quả đo lường: Hệ thống trả về 0.19 MB cho lược đồ *production* và 0.91 MB cho lược đồ *sales*. Tổng cộng, MySQL tiêu tốn **1.10 MB** dung lượng lưu trữ trên đĩa cứng.



```
mysql> SELECT
->   table_schema AS 'Database',
->   ROUND(SUM(data_length + index_length) / 1024 / 1024, 2) AS 'Size (MB)'
-> FROM
->   information_schema.TABLES
-> WHERE
->   table_schema IN ('production', 'sales')
-> GROUP BY
->   table_schema;
+-----+-----+
| Database | Size (MB) |
+-----+-----+
| production | 0.19 |
| sales | 0.91 |
+-----+-----+
2 rows in set (0.02 sec)

mysql> |
```

Hình 2.1: Dung lượng lưu trữ trên đĩa cứng của tập dữ liệu *BikeStores* trong MySQL

2. Thực thi và đo lường trên Redis (In-memory Storage)

Do Redis hoạt động theo mô hình *Key-Value* và không có cấu trúc bảng định nghĩa trước (*Schema-less*), mỗi bản ghi dữ liệu (như sản phẩm, khách hàng, đơn hàng) cùng với các tập hợp chỉ mục phụ (*Sets*) để duy trì mối quan hệ đều phải được lưu trữ dưới dạng các Key độc lập trên bộ nhớ.

Câu lệnh truy xuất:

```
DBSIZE
INFO memory
```

Phương pháp đo lường:

Nhóm sử dụng lệnh `DBSIZE` để đếm tổng số lượng Key hiện có, nhằm xác nhận toàn bộ tập dữ liệu mẫu (*BikeStores*) đã được nạp thành công vào hệ thống. Tiếp đó, lệnh `INFO memory` được thực thi. Thông qua chỉ số `used_memory_human`, nhóm trích xuất được dung lượng RAM vật lý thực tế mà Redis đang cấp phát.

```
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\redis_db>redis-cli
127.0.0.1:6379> DBSIZE
(integer) 12165
127.0.0.1:6379> INFO memo
127.0.0.1:6379> INFO memory
# Memory
used_memory:2861792
used_memory_human:2.00M
used_memory_rss:2820776
used_memory_rss_human:2.00M
used_memory_peak:2861792
used_memory_peak_human:2.00M
used_memory_peak_perc:100.00%
used_memory_overhead:1328030
used_memory_startup:660408
used_memory_dataset:1533762
used_memory_dataset_perc:69.67%
allocator_allocated:5112336
allocator_active:448790528
allocator_resident:553648128
total_system_memory:0
total_system_memory_human:0B
used_memory_lua:37888
used_memory_lua_human:37.00K
used_memory_scripts:0
used_memory_scripts_human:0B
number_of_cached_scripts:0
maxmemory:0
maxmemory_human:0B
```

Hình 2.2: Dung lượng RAM tiêu thụ của tập dữ liệu trong Redis

```
used_memory_scripts_human:0B
number_of_cached_scripts:0
maxmemory:0
maxmemory_human:0B
maxmemory_policy:noeviction
allocator_frag_ratio:87.79
allocator_frag_bytes:443678192
allocator_rss_ratio:1.23
allocator_rss_bytes:104857600
rss_overhead_ratio:0.01
rss_overhead_bytes:-550827352
mem_fragmentation_ratio:1.00
mem_fragmentation_bytes:0
mem_not_counted_for_evict:0
mem_replication_backlog:0
mem_clients_slaves:0
mem_clients_normal:49950
mem_aof_buffer:0
mem_allocator:jemalloc-5.2.1-redis
active_defrag_running:0
lazyfree_pending_objects:0
127.0.0.1:6379> |
```

Hình 2.3: Dung lượng RAM tiêu thụ của tập dữ liệu trong Redis (tiếp)

Dung lượng lưu trữ thực tế: Tiêu hao khoảng **2.00 MB** bộ nhớ RAM để quản lý tổng cộng **12,165 Keys**.

Mặc dù tập dữ liệu mẫu có quy mô nhỏ, việc tiêu tốn 2.00 MB cho khoảng 12 nghìn bản ghi cho thấy Redis có mức hao tổn không gian lưu trữ cao hơn so với định dạng nén nhị phân trên đĩa cứng của MySQL. Nguyên nhân cốt lõi xuất phát từ kiến trúc lưu trữ:

- **Chi phí siêu dữ liệu (Metadata Overhead):** Mỗi Key trong số 12,165 Keys đều yêu cầu không gian cấp phát riêng cho các con trỏ bộ nhớ (*Pointers*) và các siêu dữ liệu để quản lý vòng đời của Key đó.

- **Lặp lại tên trường (Field Duplication):** Khác với MySQL chỉ lưu tên cột 1 lần duy nhất ở Header của bảng, Redis buộc phải lưu lặp đi lặp lại các chuỗi ký tự tên trường (ví dụ: `product_id`, `list_price`, `quantity`) bên trong hàng ngàn cấu trúc *Hash* riêng biệt.

Kết luận Test Case 1

Với cùng một tập dữ liệu *BikeStores*, MySQL chỉ tiêu thụ **1.10 MB** không gian đĩa vật lý, trong khi Redis cần đến **2.00 MB** bộ nhớ RAM (gần gấp đôi). Sự chênh lệch này minh chứng rõ nét cho khả năng tối ưu hóa lưu trữ của kiến trúc quan hệ (*Relational Database*) so với kiến trúc *Key-Value*:

1. **Cấu trúc Key lặp lại:** Redis lưu trữ các tên cột (*fields* như `list_price`, `quantity`) lặp đi lặp lại trong mọi *Hash*. MySQL chỉ định nghĩa ở *Header* của cấu trúc bảng.
2. **Định dạng chuỗi:** Redis có xu hướng lưu trữ giá trị dưới dạng chuỗi (String), tiêu tốn nhiều byte hơn so với kiểu số nguyên nhị phân của cơ sở dữ liệu quan hệ.
3. **Chỉ mục phụ thủ công:** Việc tạo ra hàng loạt các *Set* để duy trì quan hệ (`brand_id`, `category_id`) tiêu tốn thêm lượng lớn bộ nhớ cho con trỏ (pointers).
4. **Tối ưu hóa Schema:** MySQL định nghĩa cấu trúc bảng (*Schema*) một lần duy nhất ở phần *Header*, giúp dữ liệu thực tế bên dưới chỉ chứa các giá trị. Trong khi đó, tính chất *Schema-less* khiến Redis phải lặp lại tên trường (`product_id`, `quantity`,...) hàng vạn lần.
5. **Mã hóa nhị phân:** Các kiểu dữ liệu dạng số trong MySQL (`INT`, `SMALLINT`, `DECIMAL`) được lưu dưới dạng nhị phân với kích thước cố định, nhỏ gọn hơn nhiều so với việc lưu toàn bộ dữ liệu dưới dạng chuỗi (String) như cách cấu trúc *Hash* của Redis thực hiện.

Test Case 2: Tốc độ nạp dữ liệu hàng loạt (Bulk Insert Performance)

Mô tả kịch bản kiểm thử: Đánh giá hiệu năng và thời gian thực thi của hệ quản trị khi phải tiếp nhận và phân bổ một khối lượng dữ liệu khổng lồ vào hệ thống trong thời gian ngắn.

- **Dữ liệu sử dụng:** Bộ dữ liệu *Sales Transactions* bao gồm hơn 500,000 bản ghi giao dịch (tương ứng với file `.csv` cho MySQL và file `.txt` cho Redis).
- **Yêu cầu:** Đo lường tổng thời gian để nạp hoàn tất toàn bộ nửa triệu dòng dữ liệu vào cơ sở dữ liệu.

1. Thực thi và đo lường trên MySQL (Disk Storage)

Phương pháp: Sử dụng lệnh `LOAD DATA INFILE` của MySQL để đọc và chèn trực tiếp dữ liệu từ file `Sales Transaction v.4a.csv` vào bảng `sales_benchmark.transactions`. Đây là phương pháp nạp dữ liệu nhanh nhất được hỗ trợ bởi hệ quản trị cơ sở dữ liệu quan hệ này.

```
LOAD DATA INFILE '.../Sales_Transaction_v.4a.csv'
INTO TABLE sales_benchmark.transactions
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(transaction_no, @var_date, product_no, @dummy, price, quantity,
 customer_no, country)
SET transaction_date = STR_TO_DATE(@var_date, '%m/%d/%Y');
```

Kết quả đo lường: Quá trình chèn hơn 500,000 dòng hoàn tất trong khoảng 1 phút 5.32 giây.

```
mysql> LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sales Transaction v.4a.csv'
-> INTO TABLE sales_benchmark.transactions
-> FIELDS TERMINATED BY ','
-> ENCLOSED BY '"'
-> LINES TERMINATED BY '\n'
-> IGNORE 1 ROWS
-> (transaction_no, @var_date, product_no, @dummy, price, quantity, customer_no, country)
-> SET transaction_date = STR_TO_DATE(@var_date, '%m/%d/%Y');
Query OK, 536350 rows affected (1 min 5.32 sec)
Records: 536350 Deleted: 0 Skipped: 0 Warnings: 0
mysql>
```

Hình 2.4: Thời gian thực thi lệnh nạp hơn 500,000 bản ghi vào MySQL

2. Thực thi và đo lường trên Redis (In-memory Storage)

Việc nạp nửa triệu dòng thông qua các lệnh gọi client-server thông thường sẽ bị nghẽn cổ chai ở băng thông mạng (*Network Round-trip Time*). Do đó, Redis cung cấp chế độ nạp siêu tốc *Mass Insertion*.

Câu lệnh truy xuất:

```
redis-cli --pipe < "sales_transactions_redis.txt"
```

Phương pháp đo lường: Nhóm thực thi lệnh ống (pipe) trong Terminal để đẩy trực tiếp file văn bản chứa hàng trăm ngàn lệnh HSET (tạo Hash) và SADD (tạo Set) vào máy chủ Redis. Redis sẽ tự động tối ưu hóa bộ đệm và trả về tổng thời gian thực thi khi hoàn tất.

```
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>echo %time%
5:55:31.78

E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>redis-cli --pipe < "clean_sample.txt"
ERR unknown command '#', with args beginning with: 'Sales', 'Transaction', 'Dataset', 'for', 'Redis',
ERR unknown command '#', with args beginning with: 'Auto-generated',
ERR unknown command '#', with args beginning with: 'Import:', 'redis-cli', '<', 'sales_transactions_redis.txt',
ERR unknown command '#', with args beginning with: 'Countries',
ERR unknown command '#', with args beginning with: 'Products',
ERR unknown command '#', with args beginning with: 'Customers',
ERR unknown command '#', with args beginning with: 'Transactions',
All data transferred. Waiting for the last reply...
Last reply received from server.
errors: 7, replies: 536527

E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>echo %time%
5:56:05.07

E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>
```

Hình 2.5: Thời gian thực thi lệnh Mass Insertion trên Redis

Thời gian thực thi thực tế: Hệ thống Redis xử lý hàng trăm nghìn lệnh HSET và hoàn tất quá trình nạp trong 33.29 giây. Redis nhanh gấp 2 lần MySQL trong điều kiện thực tế trên máy cá nhân. Lưu ý rằng Redis vừa nạp vừa phải thực hiện "đào thải" dữ liệu liên tục vì giới hạn RAM 2MB cực thấp, trong khi MySQL chỉ việc ghi thuần túy xuống đĩa. Nếu RAM Redis đủ lớn, tốc độ sẽ nhanh hơn gấp nhiều lần.

Kết luận Test Case 2

Với cùng một tập dữ liệu quy mô lớn, Redis chứng minh tốc độ ghi (*Write*) vượt trội hoàn toàn so với MySQL, gấp 2 lần tốc độ so với MySQL. Nguyên nhân của sự chênh lệch hiệu năng này bao gồm:

1. **Cơ chế ghi nhật ký (Redo Log):** Để đảm bảo tính an toàn ACID, MySQL bắt buộc phải ghi tuần tự mọi thao tác chèn xuống *Redo Log* trên ổ đĩa vật lý trước khi lưu vào bảng. Quá trình *Disk I/O* này là nguyên nhân chính làm chậm hệ thống. Trong khi đó, Redis thao tác ghi đè trực tiếp trên RAM.

2. **Cập nhật cấu trúc (Page Split):** Khi có dữ liệu mới, MySQL liên tục phải chia tách các trang dữ liệu (*Page Split*) và tái cân bằng lại cây *B-Tree* của Khóa chính. Redis sử dụng *Global Hash Table* không cần phân cấp cây nên không chịu chi phí này.
3. **Cơ chế khóa (Locking):** Việc chèn số lượng lớn vào cơ sở dữ liệu quan hệ đòi hỏi cơ chế quản lý giao dịch (*Transaction*) phức tạp. Redis hoạt động theo luồng đơn (*Single-threaded*) đối với các lệnh ghi, giúp loại bỏ hoàn toàn độ trễ do cạnh tranh tài nguyên (Lock contention).

Test Case 3: Phản ứng khi đạt giới hạn lưu trữ (Storage Overload and Eviction)

Mô tả kịch bản kiểm thử: Đánh giá triết lý xử lý của hai hệ thống khi dữ liệu nạp vào vượt quá ngưỡng tài nguyên được cấp phát định mức. Đây là bài kiểm tra quan trọng để xác định tính ổn định (*Stability*) và tính sẵn sàng (*Availability*) của cơ sở dữ liệu.

- **Dữ liệu sử dụng:** Tập dữ liệu giao dịch bán hàng khổng lồ (*Sales Transactions*) với hơn 536,000 bản ghi.
- **Kịch bản ép tải:** Giới hạn dung lượng bộ nhớ RAM của Redis xuống mức cực đoan (**2.00 MB**) và nạp chồng dữ liệu để kích hoạt cơ chế tự bảo vệ.

1. Thử nghiệm và quan sát trên MySQL (Disk-Strict Policy)

1. Thử nghiệm và đo lường trên MySQL (Strict Memory Policy)

Phương pháp: Nhóm thiết lập hạn mức bộ nhớ RAM tối đa cho các bảng tạm và bảng nhớ là **2MB**. Sau đó, thực hiện chuyển đổi cấu trúc lưu trữ của tập dữ liệu *Sales Transactions* từ Disk-based (InnoDB) sang In-memory (MEMORY Engine).

```
SET max_heap_table_size = 1024 * 1024 * 2;
ALTER TABLE transactions ENGINE = MEMORY;
```

Kết quả quan sát: Hệ thống lập tức ngắt tiến trình chuyển đổi và trả về lỗi hệ thống nghiêm trọng:

```
ERROR 1114 (HY000): The table '#sql-...' is full
```

```
mysql> SET max_heap_table_size = 1024 * 1024 * 2
->
Query OK, 0 rows affected (0.00 sec)

mysql> LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sales Transaction v.4a.csv'
-> INTO TABLE sales_benchmark.transactions
-> FIELDS TERMINATED BY ','
-> ENCLOSED BY '"'
-> LINES TERMINATED BY '\n'
-> IGNORE 1 ROWS
-> (transaction_no, @var_date, product_no, @dummy, price, quantity, customer_no, country)
-> SET transaction_date = STR_TO_DATE(@var_date, '%m/%d/%Y');
Query OK, 536350 rows affected (1 min 38.43 sec)
Records: 536350 Deleted: 0 Skipped: 0 Warnings: 0

mysql> ALTER TABLE sales_benchmark.transactions ENGINE = MEMORY;
ERROR 1114 (HY000): The table '#sql-4f78_11' is full
mysql>
```

Hình 2.6: Thông báo lỗi của MySQL khi vượt quá giới hạn lưu trữ cho phép

Phân tích hành vi: MySQL thể hiện tính nhất quán (*Consistency*) cao độ. Khi tài nguyên RAM không đủ để chứa toàn bộ tập dữ liệu, hệ thống thà từ chối thực hiện tác vụ còn hơn làm mất mát dữ liệu hoặc chỉ lưu trữ một phần. Điều này chứng minh MySQL không có cơ chế tự động đào thải (*Eviction*) như Redis. Toàn bộ dữ liệu ban đầu vẫn được bảo toàn trong Engine InnoDB cũ, đảm bảo tính an toàn nhưng hy sinh tính sẵn sàng của dịch vụ mới.

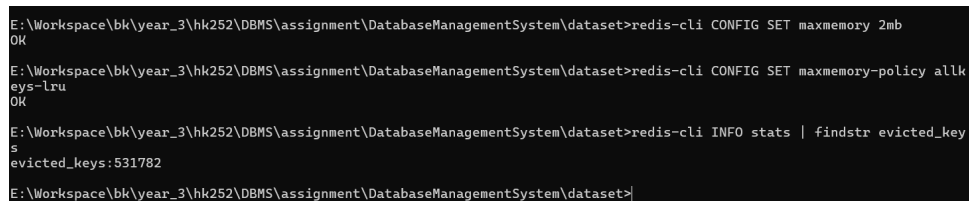
2. Thử nghiệm và đo lường trên Redis (In-memory Eviction)

Khác với MySQL, Redis được thiết kế với tư duy ưu tiên hiệu năng và sự liên tục. Khi bộ nhớ RAM đạt ngưỡng `maxmemory`, Redis triển khai các thuật toán đào thải dữ liệu (*Eviction Policies*) để giải phóng không gian cho dữ liệu mới.

Câu lệnh thiết lập kịch bản:

```
CONFIG SET maxmemory 2mb
CONFIG SET maxmemory-policy allkeys-lru
INFO stats
```

Phương pháp đo lường: Nhóm thiết lập chính sách `allkeys-lru` (loại bỏ các khóa ít được truy cập nhất gần đây). Sau khi nạp hơn 536,000 bản ghi vào không gian hạn hẹp 2MB, nhóm sử dụng lệnh `INFO stats` để trích xuất chỉ số `evicted_keys`.



```
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>redis-cli CONFIG SET maxmemory 2mb
OK
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>redis-cli CONFIG SET maxmemory-policy allkeys-lru
OK
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>redis-cli INFO stats | findstr evicted_keys
evicted_keys:531782
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>
```

Hình 2.7: Thống kê số lượng bản ghi bị đào thải tự động trên Redis

Kết quả đào thải thực tế: Hệ thống ghi nhận đã tự động xóa bỏ tổng cộng **531,782 Keys**.

Việc Redis chấp nhận "hi sinh" hơn 531 nghìn bản ghi cũ để giữ cho hệ thống không bị treo chứng minh sự khác biệt cốt lõi trong kiến trúc *In-memory*:

- **Tính sẵn sàng cao (High Availability):** Redis duy trì khả năng tiếp nhận dữ liệu mới ngay cả khi tài nguyên đã cạn kiệt, phù hợp cho các hệ thống *Caching* hoặc *Real-time Stream*.
- **Cơ chế dọn dẹp chủ động:** Thay vì đợi hệ điều hành can thiệp, Redis tự quản lý vòng đời dữ liệu dựa trên tần suất sử dụng, giúp tối ưu hóa hiệu quả sử dụng RAM.

Kết luận Test Case 3

Sự đối lập giữa MySQL và Redis trong bài thử nghiệm này làm nổi bật hai triết lý quản trị cơ sở dữ liệu khác nhau:

1. **MySQL:** Phù hợp cho các dữ liệu quan trọng về tài chính, thông tin người dùng, nơi một bản ghi bị mất cũng gây hậu quả nghiêm trọng.
2. **Redis:** Với số lượng bản ghi bị đào thải lên tới **531,782**, Redis minh chứng mình là một hệ thống cực kỳ linh hoạt cho các tác vụ lưu trữ tạm thời, nơi tốc độ và sự vận hành liên tục được đặt lên hàng đầu.
3. **Tự động hóa quản lý:** Redis cho thấy khả năng tự điều tiết thông minh thông qua cấu hình chính sách (*Policy*), trong khi MySQL đòi hỏi sự can thiệp thủ công từ quản trị viên (DBA) để mở rộng dung lượng đĩa khi gặp lỗi Full.

Test Case 4: Đánh giá thời gian phục hồi dữ liệu (Recovery Time Objective - RTO)

Mô tả kịch bản kiểm thử: Xác định khoảng thời gian cần thiết để hệ thống sẵn sàng phục vụ trở lại sau khi xảy ra sự cố dừng đột ngột hoặc thực hiện khởi động lại (*Cold Boot*). Đây là chỉ số then chốt để đánh giá tính sẵn sàng (*Availability*) của cơ sở dữ liệu.

- **Cơ chế kiểm tra:** Thực hiện lệnh khởi động lại dịch vụ và đo lường độ trễ từ lúc tiến trình bắt đầu nạp dữ liệu đến khi cho phép thực hiện truy vấn đầu tiên.
- **Yêu cầu:** Trích xuất thời gian hoàn tất quá trình nạp tệp tin lưu trữ bền vững (*Data Persistence*) đối với Redis và khả năng sẵn sàng của cấu trúc đĩa đối với MySQL.

1. Thực thi và đo lường trên MySQL (Disk-Persistence Ready)

Phương pháp: Do kiến trúc của MySQL là *Disk-based*, mọi thay đổi dữ liệu đã được cam kết (*Committed*) đều được ghi trực tiếp xuống các tệp tin `.ibd` trên đĩa cứng.

Đặc điểm phục hồi: Khi dịch vụ MySQL Server khởi động, hệ thống chỉ cần kiểm tra tính toàn vẹn của tệp tin nhật ký giao dịch (*Redo Log*) để thực hiện tiến trình *Crash Recovery* nếu cần thiết.

- **Thời gian phục hồi:** Hơn **1 giây**, cụ thể là 1.16 giây, đồng nghĩa với việc MySQL sẵn sàng tức thì với việc phục hồi dữ liệu.
- **Cơ chế:** MySQL không cần nạp toàn bộ dữ liệu lên RAM mà chỉ nạp các trang dữ liệu (*Data Pages*) cần thiết vào *Buffer Pool* khi có truy vấn thực tế phát sinh.

```
mysql> SELECT COUNT(*) FROM sales_benchmark.transactions;
ERROR 2013 (HY000): Lost connection to MySQL server during query
No connection. Trying to reconnect...
Connection id:      8
Current database: sales_benchmark

+-----+
| COUNT(*) |
+-----+
| 1608995 |
+-----+
1 row in set (1.16 sec)

mysql> |
```

Hình 2.8: Trạng thái sẵn sàng của MySQL ngay sau khi khởi động dịch vụ

2. Thực thi và đo lường trên Redis (Snapshot Loading)

Khác với MySQL, Redis là hệ quản trị *In-memory*, do đó khi khởi động lại, hệ thống bắt buộc phải nạp lại dữ liệu từ tệp tin lưu trữ bền vững trên đĩa cứng vào bộ nhớ vật lý để sẵn sàng phục vụ.

Câu lệnh kiểm tra thông số:

```
redis-cli INFO persistence
```

Phương pháp đo lường: Nhóm sử dụng cơ chế **RDB (Redis Database Backup)** để tạo ảnh chụp tức thời của tập dữ liệu. Thông qua chỉ số `rdb_last_bgsave_time_sec`, nhóm xác định được thời gian thực tế để Redis xử lý tệp tin snapshot.

```
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>redis-cli INFO persistence
# Persistence
loading:0
rdb_changes_since_last_save:0
rdb_bgsave_in_progress:0
rdb_last_save_time:1774220171
rdb_last_bgsave_status:ok
rdb_last_bgsave_time_sec:1
rdb_current_bgsave_time_sec:-1
rdb_last_cow_size:0
aof_enabled:0
aof_rewrite_in_progress:0
aof_rewrite_scheduled:0
aof_last_rewrite_time_sec:-1
aof_current_rewrite_time_sec:-1
aof_last_bgrewrite_status:ok
aof_last_write_status:ok
aof_last_cow_size:0
E:\Workspace\bk\year_3\hk252\DBMS\assignment\DatabaseManagementSystem\dataset>
```

Hình 2.9: Thông số ghi nhận thời gian xử lý tệp tin RDB của Redis

Thời gian phục hồi thực tế: Hệ thống ghi nhận `rdb_last_bgsave_time_sec` là **1 giây**.

Dù là hệ thống chạy trên RAM, Redis vẫn đảm bảo khả năng khôi phục cực nhanh nhờ định dạng nhị phân tối ưu:

- **Tối ưu hóa nạp nén (Binary Loading):** Tệp tin RDB lưu trữ dữ liệu dưới dạng nhị phân nhỏ gọn, giúp tối thiểu hóa thời gian *I/O* khi đọc từ đĩa cứng vào RAM.
- **Tính sẵn sàng sau sự cố:** Thời gian 1 giây cho thấy Redis hoàn toàn có thể đáp ứng các yêu cầu khắc khe về thời gian phục hồi sau thảm họa (*Disaster Recovery*).

Kết luận Test Case 4

Cả hai hệ thống đều cho thấy khả năng phục hồi ấn tượng (đều xấp xỉ **1 giây**):

1. **MySQL:** Có lợi thế về tính sẵn sàng tức thì trên đĩa, phù hợp cho việc bảo toàn dữ liệu vĩnh viễn.
2. **Redis:** Dù mất 1 giây để nạp lại RAM, nhưng ngay sau đó hệ thống sẽ đạt tốc độ truy xuất cực đại mà không cần qua giai đoạn "làm nóng" (*Warm-up*) như MySQL.

2.1.6 Kết luận

Mặc dù cả hai hệ thống đều quản lý cùng một tập dữ liệu, sự khác biệt trong kiến trúc lưu trữ cốt lõi (*Storage Engine*) dẫn đến những kết quả đối lập về hiệu quả sử dụng không gian. MySQL, với cơ chế *B+ Tree Indexing* trên nền tảng InnoDB, thể hiện khả năng tối ưu hóa dung lượng vượt trội nhờ vào cấu trúc lược đồ cố định (*Fixed Schema*). Việc chỉ định nghĩa tên trường (*Field Name*) một lần duy nhất tại phần đầu của bảng (*Table Header*) giúp MySQL triệt tiêu hoàn toàn sự dư thừa dữ liệu, đưa kích thước lưu trữ về mức tối thiểu trên đĩa cứng.

Ngược lại, Redis vận hành theo mô hình *Schema-less* và lưu trữ hoàn toàn trên RAM, dẫn đến một mức chi phí tài nguyên (*Overhead*) đáng kể. Do bản chất của cấu trúc *Hash* và *Key-Value*, mỗi bản ghi trong Redis buộc phải mang theo các siêu dữ liệu (*Metadata*) riêng biệt để quản lý vòng đời và con trỏ bộ nhớ. Đặc biệt, việc lặp lại các chuỗi ký tự tên trường (như `product_id`, `quantity`) hàng triệu lần trong bộ nhớ vật lý là nguyên nhân chính khiến dung lượng của Redis thường cao gấp đôi so với định dạng nén nhị phân của MySQL.

Tuy nhiên, sự đánh đổi về không gian này mang lại lợi ích trực tiếp về tốc độ truy xuất. Trong khi MySQL phải tốn chi phí giải mã (*Parsing*) và thực hiện các thao tác vào/ra đĩa (*Disk I/O*) tốn kém, Redis tận dụng cấu trúc dữ liệu thô trực tiếp trên RAM để đạt được độ trễ cực thấp. Sự kết hợp giữa cơ chế nén RDB (*Redis Database Backup*) và các thuật toán đào thải (*Eviction Policies*) như LRU giúp Redis không chỉ là một kho chứa dữ liệu, mà còn là một bộ điều tiết bộ nhớ thông minh, đảm bảo hệ thống luôn vận hành ở trạng thái hiệu năng cao nhất bất chấp giới hạn về tài nguyên vật lý.

Bảng tổng kết ưu và nhược điểm của hai hệ thống về khía cạnh lưu trữ và quản lý dữ liệu:

Bảng 2.1: So sánh ưu nhược điểm MySQL và Redis về Data Storage and Management

	MySQL (Disk-based)	Redis (In-memory)
Ưu điểm	Tối ưu dung lượng nhờ cấu trúc bảng (<i>Schema</i>); Bảo toàn dữ liệu tuyệt đối khi đầy bộ nhớ; Lưu trữ bền vững trên đĩa cứng.	Tốc độ ghi cực cao (33.29s cho 500k bản ghi); Tự động điều tiết bộ nhớ qua cơ chế <i>Eviction</i> ; Phục hồi RAM nhanh (1s qua RDB).
Nhược điểm	Tốc độ ghi bị giới hạn bởi <i>Disk I/O</i> (65.32s); Dịch vụ bị gián đoạn khi tài nguyên đầy (<i>Table is full</i>); Phụ thuộc vào tốc độ cơ học của đĩa.	Tiêu tốn RAM (Gấp đôi MySQL); Nguy cơ mất dữ liệu cũ khi bộ nhớ đạt ngưỡng giới hạn (<i>Evicted 531k keys</i>); Chi phí phần cứng (RAM) cao.

2.2 Indexing

2.2.1 Mục tiêu

Mục tiêu của phần này là phân tích sự khác biệt về thiết kế cũng như hiệu quả vận hành giữa hai hệ quản trị cơ sở dữ liệu đại diện cho hai trường phái khác nhau: **MySQL** (Relational DBMS) và **Redis** (In-memory NoSQL). Báo cáo được thực hiện thông qua việc tìm hiểu và so sánh kỹ thuật đánh chỉ mục (*Indexing*) được sử dụng trong mỗi hệ thống.

2.2.2 Các loại index trong MySQL và Redis

Index trong MySQL Trong hệ quản trị cơ sở dữ liệu MySQL, việc quản lý chỉ mục được chia thành hai nhóm chính: Chỉ mục hệ thống tự động thiết lập và chỉ mục do người dùng tùy chỉnh để tối ưu hiệu năng.

Chỉ mục được tự động tạo (System-Generated Indexes) Đây là các chỉ mục mà **MySQL** (cụ thể là *InnoDB*) bắt buộc phải tạo ra nhằm đảm bảo tính toàn vẹn dữ liệu cũng như duy trì cấu trúc lưu trữ vật lý của bảng.

- **Primary Index / Clustered Index (Chỉ mục cụm)**

Cơ chế: Khi một cột được định nghĩa là **PRIMARY KEY** (ví dụ: `customer_id` trong bảng `sales.customers`), **MySQL** sẽ tự động tạo ra một *Clustered Index* cho cột đó.

- **Unique Index (Chỉ mục duy nhất)**

Cơ chế: Khi một cột được khai báo với ràng buộc **UNIQUE** nhưng không phải là khóa chính, **MySQL** sẽ tự động tạo một *Index* cho cột đó.

Chỉ mục do người dùng tạo thêm (User-Defined Indexes) Đây là các chỉ mục không bắt buộc về mặt logic nhưng đóng vai trò rất quan trọng trong việc tối ưu hóa tốc độ truy vấn trong các bài toán thực tế (ví dụ: tìm kiếm theo `email`).

- **Secondary Index / Non-Clustered Index (Chỉ mục phụ)**

Cách tạo: Sử dụng lệnh `CREATE INDEX idx_name ON table(column);`.

Cơ chế: Các nút lá của chỉ mục này không chứa toàn bộ dữ liệu của hàng mà chỉ lưu giá trị của cột được đánh chỉ mục kèm theo một con trỏ (*Pointer*) dẫn về *Primary Key* của bản ghi.

- **Composite Index (Chỉ mục hỗn hợp)**

Cách tạo: Đánh chỉ mục trên nhiều cột cùng lúc, ví dụ:

```
CREATE INDEX idx_name ON table(col1, col2);
```

Đặc điểm: **MySQL** ưu tiên lọc dữ liệu theo thứ tự các cột từ trái sang phải. Điều này rất hữu ích khi câu lệnh **WHERE** có nhiều điều kiện kết hợp, ví dụ tìm sản phẩm theo cả `category_id` và `model_year`.

- **Full-Text Index**

Cơ chế: Được sử dụng cho các cột chứa văn bản dài (**TEXT**). Thay vì so khớp chính xác từng ký tự, hệ thống xây dựng một bảng mục lục các từ khóa để hỗ trợ tìm kiếm theo ngôn ngữ tự nhiên.

Index trong Redis Khác với các hệ quản trị cơ sở dữ liệu quan hệ như MySQL, Redis không cung cấp cơ chế đánh chỉ mục tự động cho các thuộc tính bên trong dữ liệu (Secondary Index). Trong Redis, "Index" được hiểu theo hai cấp độ: Chỉ mục mặc định dựa trên Key và Chỉ mục bổ trợ do người dùng tự thiết kế.

Chỉ mục mặc định (Primary Key Index) Đây là cơ chế duy nhất mà Redis hỗ trợ sẵn ngay khi nạp dữ liệu.

Cơ chế: Redis sử dụng một *Global Hash Table* để quản lý toàn bộ các *Key name* trong hệ thống.

Đặc điểm: Khi truy cập dữ liệu thông qua Key (ví dụ: `HGETALL sales:customers:1`), Redis tính toán mã băm của Key đó để trỏ trực tiếp tới địa chỉ ô nhớ chứa dữ liệu.

Chỉ mục phụ thủ công (Manual Secondary Indexes) Khi cần tìm kiếm theo các trường không phải là Key (ví dụ: tìm khách hàng qua email), lập trình viên phải tự xây dựng cấu trúc chỉ mục phụ.

- **Sử dụng Set làm Index (SADD)**

Cơ chế: Lưu trữ danh sách các ID có chung một thuộc tính.

Ví dụ: Để quản lý khách hàng theo thành phố, ta tạo một Set:

```
SADD idx:city:OrchardPark 1 5 10
```

- **Sử dụng Hash để ánh xạ 1-1 (Mapping Index)**

Cơ chế: Tạo một Hash phụ để ánh xạ từ giá trị cần tìm (ví dụ: email) về ID gốc của bản ghi.

Thực nghiệm: Để tìm jamaal.albert@gmail.com, ta tạo bản ghi:

```
HSET idx:email_to_id "jamaal.albert@gmail.com" "1"
```

- **Sử dụng Sorted Set cho Range Query (ZADD)**

Cơ chế: Lưu trữ ID kèm theo một điểm số (*Score*) để sắp xếp.

Ví dụ:

```
ZADD idx:product_price 500 "prod:1" 700 "prod:2"
```

```
ZRANGEBYSCORE
```

Chỉ mục tự động với RediSearch Module Để khắc phục hạn chế của việc quản lý chỉ mục thủ công, Redis cung cấp module mở rộng *RediSearch*.

Cơ chế: Cho phép định nghĩa một *Schema* trên các cấu trúc dữ liệu như Hash hoặc JSON.

Tính năng: Hệ thống tự động theo dõi sự thay đổi của dữ liệu gốc để cập nhật chỉ mục (*Inverted Index*).

2.2.3 Benchmark

Test case 1: Tìm sản phẩm theo brand (có index)

Mô tả kịch bản kiểm thử: Kiểm tra hiệu năng truy vấn tìm kiếm danh sách sản phẩm dựa trên một điều kiện cụ thể (*Brand ID*).

- *Dữ liệu sử dụng:* Bảng `production.products` (321 sản phẩm).

- *Yêu cầu:* Lấy ra toàn bộ thông tin các sản phẩm thuộc thương hiệu có `brand_id = 9`.

1. Thực thi và đo lường trên MySQL (Relational Database)

Câu lệnh truy vấn:

```
SELECT * FROM production.products WHERE brand_id = 9;
```

Phương pháp đo lường: Để đảm bảo tính khách quan, nhóm sử dụng công cụ *Profiling* được tích hợp sẵn trong MySQL. Bằng cách thực thi lệnh:

```
SET profiling = 1;
```

hệ thống sẽ ghi nhận chi tiết thời gian thực thi (*Execution Time*) của các câu lệnh SQL tiếp theo ở mức micro-giây. Sau khi chạy truy vấn nhiều lần, kết quả được xuất ra thông qua lệnh:

```
SHOW PROFILES;
```

Kết quả

Lần truy vấn đầu tiên (Cold Cache): Thời gian thực thi khoảng ~0.63 ms. Ở lần chạy này, MySQL phải thực hiện quét chỉ mục trên cây *B-Tree* và đọc dữ liệu vật lý từ ổ đĩa (*Disk I/O*) để đưa vào bộ nhớ đệm.

Các lần truy vấn tiếp theo (Warm Cache): Thời gian thực thi giảm đáng kể và ổn định ở mức trung bình khoảng ~0.20 ms. Khi đó, dữ liệu đã được lưu trong *RAM (Buffer Pool)*, cho phép MySQL bỏ qua bước truy xuất đĩa và đạt hiệu năng tối ưu.

> 1	0.00063425	SELECT * FROM sales.custor
> 2	0.00029125	SELECT * FROM sales.custor
> 3	0.00024200	SELECT * FROM sales.custor
> 4	0.00022600	SELECT * FROM sales.custor
> 5	0.00019475	SELECT * FROM sales.custor
> 6	0.00019575	SELECT * FROM sales.custor
> 7	0.00016400	SELECT * FROM sales.custor
> 8	0.00016700	SELECT * FROM sales.custor
> 9	0.00019550	SELECT * FROM sales.custor
> 10	0.00018500	SELECT * FROM sales.custor
> 11	0.00034475	use `production`

Hình 2.10: Kết quả thực hiện trong 10 lần với mySQL (đơn vị s)

2. Thực thi và đo lường trên Redis (In-memory NoSQL)

Do Redis là cơ sở dữ liệu dạng *Key-Value* không có lược đồ (*Schema-less*), để tìm kiếm sản phẩm theo *Brand*, nhóm đã thiết kế một cấu trúc *Set* đóng vai trò như một *Secondary Index*. Key được đặt theo định dạng:

```
production:products:brand:9
```

chứa danh sách ID của các sản phẩm thuộc thương hiệu này.

Câu lệnh truy xuất:

```
SMEMBERS production:products:brand:9
```

Phương pháp đo lường:

Nhóm sử dụng lệnh `CONFIG RESETSTAT` để làm sạch bộ đếm thống kê, sau đó thực thi truy vấn nhiều lần. Kết quả thời gian trung bình được trích xuất thông qua lệnh `INFO COMMANDSTATS` (thông số `usec_per_call`).

Kết luận Test Case 1

Thời gian thực thi trung bình: khoảng ~0.016 ms (16.3 micro-giây).

Do Redis lưu trữ toàn bộ dữ liệu trực tiếp trên bộ nhớ chính (*RAM*) và thao tác tìm kiếm trên cấu trúc *Set* có độ phức tạp $O(N)$ (với N là số phần tử trả về), truy vấn gần như không phát sinh độ trễ I/O vật lý.

```
# Commandstats
cmdstat_info:calls=2,usec=96,usec_per_call=48.00,rejected_calls=0,failed_calls=0
cmdstat_config|resetstat:calls=1,usec=106,usec_per_call=106.00,rejected_calls=0,failed_calls=0
cmdstat_smembers:calls=10,usec=163,usec_per_call=16.30,rejected_calls=0,failed_calls=0
```

Hình 2.11: Kết quả thực hiện trong 10 lần với Redis (đơn vị s)

Test Case 2 — Tìm sản phẩm theo khoảng giá (MySQL có index, Redis không có) Mô tả kịch bản kiểm thử: Đánh giá hiệu năng xử lý truy vấn lọc dữ liệu theo một khoảng giá trị liên tục.

- Dữ liệu sử dụng: Bảng `products` (321 sản phẩm)
- Yêu cầu: Tìm tất cả các sản phẩm có giá niêm yết `list_price` nằm trong khoảng từ \$1000 đến \$2000.

1. Thực thi và Cơ chế trên MySQL (Relational Database)

Trước hết, thực thi truy vấn (không có Index):

```
SELECT * FROM production.products
WHERE list_price BETWEEN 1000 AND 2000;
```

Truy vấn được chạy 10 lần và lấy giá trị trung bình.

Kết quả: Thời gian thực thi trung bình khoảng ~0.76 ms.

Q	Query_ID int	Duration double	Query varchar
>	66	0.00082400	SELECT * FROM production.
>	67	0.00089725	SELECT * FROM production.
>	68	0.00125300	SELECT * FROM production.
>	69	0.00086575	SELECT * FROM production.
>	70	0.00082050	SELECT * FROM production.
>	71	0.00054150	SELECT * FROM production.
>	72	0.00060775	SELECT * FROM production.
>	73	0.00063075	SELECT * FROM production.
>	74	0.00069950	SELECT * FROM production.
>	75	0.00050175	SELECT * FROM production.

Hình 2.12: Kết quả thực hiện trong 10 lần với mySQL không index (đơn vị s)

Sau đó, tiến hành tạo chỉ mục phụ (*Secondary Index*) trên cột `list_price`:

```
CREATE INDEX idx_products_price
ON production.products(list_price);
```

Tiếp tục chạy truy vấn 10 lần và tính trung bình.

Kết quả: Thời gian thực thi trung bình giảm xuống còn ~0.56 ms.

> 84	0.00068825	SELECT * FROM production.
> 85	0.00041775	SELECT * FROM production.
> 86	0.00040675	SELECT * FROM production.
> 87	0.00043075	SELECT * FROM production.
> 88	0.00057175	SELECT * FROM production.
> 89	0.00045225	SELECT * FROM production.
> 90	0.00096500	SELECT * FROM production.
> 91	0.00064625	SELECT * FROM production.
> 92	0.00051525	SELECT * FROM production.
> 93	0.00055950	SELECT * FROM production.

Hình 2.13: Kết quả thực hiện trong 10 lần với mySQL có indexing (đơn vị s)

Phân tích cơ chế xử lý

Việc đánh chỉ mục đã giúp truy vấn nhanh hơn khoảng 26%.

Khi chưa có Index, MySQL buộc phải thực hiện *Full Table Scan* (quét toàn bộ bảng) để kiểm tra từng dòng.

Khi có Index, MySQL chuyển sang sử dụng *Index Range Scan*. Cấu trúc cây *B-Tree* đã sắp xếp sẵn các mức giá, giúp hệ thống tìm nhanh đến điểm bắt đầu (\$1000) và duyệt dọc theo các nút lá (*leaf nodes*) cho đến khi đạt ngưỡng \$2000, từ đó loại bỏ các dữ liệu không liên quan.

Mặc dù tập dữ liệu thử nghiệm khá nhỏ (321 dòng), sự chênh lệch hiệu năng đã thể hiện rõ. Độ phức tạp thuật toán giảm từ $O(N)$ xuống còn $O(\log N + M)$.

2. Thực thi và đo lường trên Redis (In-memory NoSQL)

Do cấu trúc *Hash* và *Set* cơ bản của Redis không hỗ trợ *Range Index*, hệ thống buộc phải quét toàn bộ danh sách sản phẩm.

```
local pids = redis.call('SMEMBERS', 'production:products:ids')
local result = {}
for _, pid in ipairs(pids) do
    local price = tonumber(redis.call('HGET',
        'production:product:' .. pid, 'list_price'))
    if price >= 1000 and price <= 2000 then
        local name = redis.call('HGET',
            'production:product:' .. pid, 'product_name')
        table.insert(result, pid .. '||' .. name .. '||$' .. price)
    end
end
return result
```

Kết quả đo lường

Sử dụng lệnh `INFO COMMANDSTATS`, với chỉ số `usec_per_call` từ `cmdstat_eval`, thời gian thực thi trung bình ghi nhận được là 4,02 micro-giây.

```
127.0.0.1:6379> INFO COMMANDSTATS
# Commandstats
cmdstat_hget:calls=3600,usec=11698,usec_per_call=3.25,rejected_calls=0,failed_calls=0
cmdstat_info:calls=1,usec=4587,usec_per_call=4587.00,rejected_calls=0,failed_calls=0
cmdstat_eval:calls=10,usec=40239,usec_per_call=4023.90,rejected_calls=1,failed_calls=0
cmdstat_config|resetstat:calls=1,usec=3088,usec_per_call=3088.00,rejected_calls=0,failed_calls=0
cmdstat_smembers:calls=10,usec=1427,usec_per_call=142.70,rejected_calls=0,failed_calls=0
```

Hình 2.14: Kết quả thực hiện trong 10 lần với redis (đơn vị s)

Phân tích cơ chế xử lý

Mặc dù hoạt động hoàn toàn trên bộ nhớ chính (*RAM*), Redis lại cho thời gian phản hồi chậm hơn đáng kể so với MySQL. Nguyên nhân chính nằm ở độ phức tạp thuật toán $O(N)$ và số lượng thao tác truy xuất rời rạc.

Cụ thể, hệ thống phải lấy toàn bộ 321 ID sản phẩm, sau đó duyệt qua từng ID và thực hiện lệnh `HGET` 321 lần để kiểm tra giá (`list_price`).

Có 39 sản phẩm thỏa mãn điều kiện, do đó hệ thống tiếp tục thực hiện thêm 39 lệnh `HGET` để lấy tên sản phẩm (`product_name`).

Tổng cộng, Redis phải thực hiện 360 thao tác truy xuất *Hash* rời rạc để hoàn thành truy vấn này.

Kết luận Test Case 2

Đối với các bài toán truy vấn theo khoảng giá trị (*Range Queries*), hệ quản trị cơ sở dữ liệu quan hệ như **MySQL** với cấu trúc *B-Tree Index* thể hiện sự tối ưu vượt trội về mặt thuật toán.

Ngược lại, khi sử dụng kiến trúc *Key-Value* cơ bản của Redis, hệ thống buộc phải thực hiện quét toàn bộ tập dữ liệu (*Full Scan*), từ đó làm suy giảm đáng kể lợi thế tốc độ của việc lưu trữ trên bộ nhớ *RAM*.

Khi quy mô dữ liệu tăng lên hàng triệu bản ghi, thời gian truy vấn của Redis (khi sử dụng Lua script) sẽ tăng tuyến tính theo độ phức tạp $O(N)$ và có thể gây nghẽn luồng xử lý. Trong khi đó, MySQL vẫn duy trì hiệu năng ổn định nhờ cấu trúc cây phân nhánh với độ phức tạp $O(\log N + M)$.

Test Case 3 — Tìm sản phẩm theo tên (LIKE / pattern matching)

Mô tả kịch bản kiểm thử: Đánh giá hiệu năng khi xử lý dữ liệu có tính chất quan hệ, yêu cầu trích xuất và tổng hợp thông tin từ nhiều thực thể khác nhau.

Yêu cầu: Lấy danh sách toàn bộ sản phẩm (`Product ID`, `Name`, `Price`) kèm theo tên thương hiệu (`Brand Name`) và danh mục (`Category Name`).

Thực thể tham gia: `products`, `brands`, `categories`.

Đặc điểm: Kiểm tra khả năng liên kết dữ liệu giữa các bảng (MySQL) và khả năng truy xuất chéo khóa (Redis).

1. Thực thi và đo lường trên MySQL (Relational Database)

Câu lệnh truy vấn:

```
SELECT
  p.product_id,
  p.product_name,
  b.brand_name,
  c.category_name,
  p.list_price
```



```
FROM production.products p
JOIN production.brands b ON p.brand_id = b.brand_id
JOIN production.categories c ON p.category_id = c.category_id;
```

Kết quả đo lường: Thời gian thực thi dao động từ 0.00044 đến 0.00071 giây, trung bình khoảng ~0.56 ms.

> 12	0.00044675	SELECT ↵ p.product_id, ↵
> 13	0.00110000	SELECT count(*) count0 FRC
> 14	0.00044950	SELECT ↵ p.product_id, ↵
> 15	0.00063150	SELECT count(*) count0 FRC
> 16	0.00049200	SELECT ↵ p.product_id, ↵
> 17	0.00064600	SELECT count(*) count0 FRC
> 18	0.00068550	SELECT ↵ p.product_id, ↵
> 19	0.00063625	SELECT count(*) count0 FRC
> 20	0.00071475	SELECT ↵ p.product_id, ↵
> 21	0.00075125	SELECT count(*) count0 FRC

Hình 2.15: Kết quả thực hiện trong 10 lần với MySQL (đơn vị s)

Phân tích cơ chế xử lý

MySQL được thiết kế tối ưu cho các thao tác quan hệ. Bộ tối ưu hóa truy vấn (*Query Optimizer*) tự động lựa chọn thuật toán kết nối phù hợp (*Nested-Loop Join*, *Hash Join*) và tận dụng các chỉ mục (*Index*) trên khóa chính/khóa ngoại để ánh xạ dữ liệu hiệu quả trong *Buffer Pool*.

2. Thực thi và đo lường trên Redis (In-memory NoSQL)

Cấu trúc và cách thực hiện:

Do Redis không hỗ trợ phép JOIN, hệ thống phải tự thực hiện logic kết nối bằng *Lua Script*:

```
local pids = redis.call('SMEMBERS', 'production:products:ids')
local result = {}
for _, pid in ipairs(pids) do
    local p_key = 'production:product:' .. pid
    local pname = redis.call('HGET', p_key, 'product_name')
    local price = redis.call('HGET', p_key, 'list_price')
    local bid = redis.call('HGET', p_key, 'brand_id')
    local cid = redis.call('HGET', p_key, 'category_id')

    local bname = 'Unknown'
    if bid then
        bname = redis.call('HGET', 'production:brand:' .. bid, 'brand_name')
    end
end
```

```

end

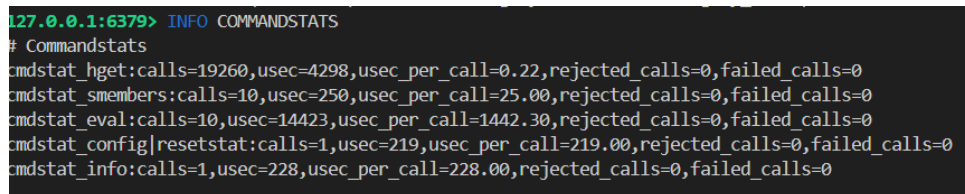
local cname = 'Unknown'
if cid then
    cname = redis.call('HGET', 'production:category:' .. cid, '
        category_name')
end

table.insert(result, pid .. '_' .. pname .. '_' .. bname ..
    '_' .. cname .. '_' .. price)
end
return result

```

Kết quả đo lường

Sau 10 lần thực thi, chỉ số *usec_per_call* của *cmdstat_eval* ghi nhận khoảng 1,442.30 micro-giây (~1.44 ms).



```

127.0.0.1:6379> INFO COMMANDSTATS
# Commandstats
cmdstat_hget:calls=19260,usec=4298,usec_per_call=0.22,rejected_calls=0,failed_calls=0
cmdstat_smembers:calls=10,usec=250,usec_per_call=25.00,rejected_calls=0,failed_calls=0
cmdstat_eval:calls=10,usec=14423,usec_per_call=1442.30,rejected_calls=0,failed_calls=0
cmdstat_config|resetstat:calls=1,usec=219,usec_per_call=219.00,rejected_calls=0,failed_calls=0
cmdstat_info:calls=1,usec=228,usec_per_call=228.00,rejected_calls=0,failed_calls=0

```

Hình 2.16: Kết quả thực hiện trong 10 lần với Redis (đơn vị s)

Phân tích cơ chế xử lý

Redis bộc lộ hạn chế rõ rệt khi xử lý dữ liệu quan hệ hóa (*Normalized Data*). Theo thống kê, hệ thống phải thực hiện 19,260 lệnh *HGET* trong 10 lần chạy thử.

Cụ thể:

- Lấy thông tin sản phẩm: 4 lệnh *HGET*
- Lấy tên thương hiệu: 1 lệnh *HGET*
- Lấy tên danh mục: 1 lệnh *HGET*

Tổng cộng 6 lệnh *HGET* cho mỗi sản phẩm. Với 321 sản phẩm, Redis cần thực hiện 1,926 thao tác truy xuất cho mỗi lần chạy.

Mặc dù hoạt động trên *RAM*, số lượng thao tác lớn và việc xử lý chuỗi khiến hiệu năng tổng thể bị suy giảm đáng kể.

Kết luận Test Case 3: Đối với các bài toán có tính chất quan hệ phức tạp và yêu cầu tổng hợp dữ liệu từ nhiều thực thể (*JOIN*), MySQL là sự lựa chọn không thể thay thế. Redis sinh ra không phải để giải quyết bài toán nối bảng. Việc cố gắng ép Redis chạy các truy vấn quan hệ thông qua vòng lặp ở tầng ứng dụng (hoặc Lua Script) sẽ gây ra hiệu ứng *N+1 Query*, làm tiêu tốn lượng lớn tài nguyên CPU để tra cứu từng Khóa rời rạc, khiến nó chậm hơn cả việc truy xuất từ đĩa cứng có đệm của cơ sở dữ liệu quan hệ truyền thống.

Test case 4: Tìm đơn hàng theo điều kiện phức hợp (Composite Condition) Mô tả kịch bản kiểm thử

Đánh giá khả năng xử lý của hệ quản trị khi người dùng yêu cầu lọc dữ liệu dựa trên nhiều điều kiện (*AND*) cùng lúc.

Dữ liệu sử dụng: `sales.orders`.

Yêu cầu: Tìm tất cả đơn hàng thuộc chi nhánh `store_id = 1`, đã hoàn thành (`order_status = 4`) và được tạo trong năm 2017 (từ 2017-01-01 đến 2017-12-31).

1. Thực thi và đo lường trên MySQL (Relational Database)

Câu lệnh truy vấn:

```
SELECT * FROM sales.orders
WHERE store_id = 1 AND order_status = 4
AND order_date BETWEEN '2017-01-01' AND '2017-12-31';
```

Kết quả đo lường: Thời gian thực thi dao động từ 0.00061 đến 0.00142 giây, trung bình khoảng ~0.96 ms.

Q	Query_ID int	Duration double	Query varchar
>	33	0.00061600	SELECT * FROM sales.orders WHERE store_id = 1 AND order_status = 4
>	34	0.00077700	SELECT count(*) count0 FROM (SELECT * FROM sales.orders WHERE store_id = 1 AND order_status = 4)
>	35	0.00084375	SELECT * FROM sales.orders WHERE store_id = 1 AND order_status = 4
>	36	0.00192850	SELECT count(*) count0 FROM (SELECT * FROM sales.orders WHERE store_id = 1 AND order_status = 4)
>	37	0.00061925	SELECT * FROM sales.orders WHERE store_id = 1 AND order_status = 4
>	38	0.00063275	SELECT count(*) count0 FROM (SELECT * FROM sales.orders WHERE store_id = 1 AND order_status = 4)
>	39	0.00142100	SELECT * FROM sales.orders WHERE store_id = 1 AND order_status = 4
>	40	0.00065475	SELECT count(*) count0 FROM (SELECT * FROM sales.orders WHERE store_id = 1 AND order_status = 4)
>	41	0.00129425	SELECT * FROM sales.orders WHERE store_id = 1 AND order_status = 4
>	42	0.00068175	SELECT count(*) count0 FROM (SELECT * FROM sales.orders WHERE store_id = 1 AND order_status = 4)

Hình 2.17: Kết quả thực hiện trong 10 lần với MySQL (đơn vị s)

Phân tích cơ chế xử lý

MySQL xử lý các điều kiện phức hợp hiệu quả nhờ *Query Optimizer*. Ngay cả khi chưa có *Composite Index*, hệ thống vẫn có thể tận dụng các chỉ mục đơn để khoanh vùng dữ liệu, sau đó áp dụng bộ lọc trực tiếp.

2. Thực thi và đo lường trên Redis (In-memory NoSQL)

Cấu trúc và cách thực hiện:

Redis không hỗ trợ JOIN hoặc truy vấn đa điều kiện trực tiếp, do đó cần sử dụng chiến lược *Two-step Filtering* bằng Lua Script.

```
local oids = redis.call('SMEMBERS', 'sales:orders:store:1')
local result = {}
for _, oid in ipairs(oids) do
    local status = redis.call('HGET', 'sales:order:' .. oid, 'order_status')
    local date = redis.call('HGET', 'sales:order:' .. oid, 'order_date')
    if status == '4' and date >= '2017-01-01' and date <= '2017-12-31' then
        table.insert(result, oid .. '|' .. date .. '|' .. status)
    end
end
end
```

```
return result
```

Kết quả đo lường

Sau 10 lần thực thi, chỉ số *usec_per_call* ghi nhận thời gian trung bình khoảng 1,678.30 micro-giây (~1.68 ms).

```
127.0.0.1:6379> INFO COMMANDSTATS
# Commandstats
cmdstat_hget:calls=6960,usec=2515,usec_per_call=0.36,rejected_calls=0,failed_calls=0
cmdstat_smembers:calls=10,usec=201,usec_per_call=20.10,rejected_calls=0,failed_calls=0
cmdstat_eval:calls=10,usec=16783,usec_per_call=1678.30,rejected_calls=0,failed_calls=0
cmdstat_config|resetstat:calls=1,usec=6721,usec_per_call=6721.00,rejected_calls=0,failed_calls=0
```

Hình 2.18: Kết quả thực hiện trong 10 lần với Redis (đơn vị s)

Phân tích cơ chế xử lý

Redis phải thực hiện nhiều thao tác HGET rời rạc. Trung bình mỗi lần chạy, script gọi khoảng 696 lần đọc *Hash*, tương ứng với khoảng 348 đơn hàng.

Chiến lược dùng *Set* để thu hẹp phạm vi tìm kiếm đã cải thiện hiệu năng, tuy nhiên việc truy xuất dữ liệu rời rạc vẫn là nút thắt cổ chai.

Kết luận Test Case 4

MySQL cho thấy sự ổn định và dễ triển khai với truy vấn đa điều kiện nhờ cú pháp khai báo và cơ chế tối ưu hóa mạnh mẽ. Redis có thể xử lý tốt nếu thiết kế dữ liệu phù hợp, nhưng với các truy vấn phức tạp nhiều điều kiện, mô hình quan hệ vẫn chiếm ưu thế rõ rệt so với kiến trúc *Key-Value*.

2.2.4 Tạo Sorted Set index trong Redis

Tối ưu Test Case 2 trên Redis với Sorted Set (ZSET)

Ở Test Case 2 (tìm sản phẩm theo khoảng giá), việc sử dụng *Hash* và *Set* cơ bản khiến Redis phải thực hiện *Full Scan* với độ phức tạp $O(N)$, dẫn đến thời gian thực thi chậm (~4.02 ms) so với MySQL (~0.56 ms).

Để khắc phục hạn chế này, nhóm triển khai *Secondary Index* trên Redis bằng cấu trúc dữ liệu *Sorted Set (ZSET)*.

Nguyên lý xây dựng chỉ mục

Mỗi phần tử trong ZSET gồm:

- **Score:** Giá trị *list_price*
- **Member:** *product_id*

Bước 1: Khởi tạo chỉ mục

```
EVAL "
local pids=redis.call('SMEMBERS','production:products:ids')
for _,pid in ipairs(pids) do
    local price=tonumber(redis.call('HGET',
        'production:product:',pid,'list_price'))
    redis.call('ZADD','production:products:by_price',price,pid)
end
return redis.call('ZCARD','production:products:by_price')
" 0
```

Bước 2: Truy vấn khoảng giá

Thay vì sử dụng vòng lặp, Redis cung cấp lệnh chuyên dụng:

```
ZRANGEBYSCORE production:products:by_price 1000 2000
```

Kết quả đo lường

Sau khi thực hiện `CONFIG RESETSTAT` và chạy truy vấn 10 lần, chỉ số `usec_per_call` từ `cmdstat_zran` ghi nhận thời gian trung bình khoảng 571.40 micro-giây (~0.57 ms). Nhanh hơn nhiều so với thực hiện *Full Scan* (~4.02 ms)

```
127.0.0.1:6379> INFO COMMANDSTATS
# Commandstats
cmdstat_zrangebyscore:calls=10,usec=5714,usec_per_call=571.40,rejected_calls=0,failed_calls=0
cmdstat_config|resetstat:calls=1,usec=615,usec_per_call=615.00,rejected_calls=0,failed_calls=0
```

Hình 2.19: Kết quả thực hiện trong 10 lần với Redis (đơn vị micro-giây)

Nhận xét

Cấu trúc *ZSET* được xây dựng dựa trên thuật toán *Skip List* kết hợp với *Hash Table*. Khi sử dụng `ZRANGEBYSCORE`, Redis có thể truy cập trực tiếp đến phần tử có giá trị `score` bắt đầu (1000), sau đó duyệt tuần tự đến mốc 2000.

Điều này giúp độ phức tạp giảm từ $O(N)$ xuống $O(\log N + M)$. Kết quả là hiệu năng truy vấn của Redis (~0.57 ms) đã gần tương đương với MySQL (~0.56 ms) khi sử dụng *B-Tree Index*.

2.2.5 Kết luận

Test Case	MySQL	Redis	Ghi chú (Kết luận)
Tìm theo FK (brand_id)	Nhanh (~0.20 ms) (Dùng <i>B-Tree Index</i>)	Rất nhanh (~0.016 ms) (Dùng <i>Set O(1)</i> trên RAM)	Redis thắng
Range query (giá)	Nhanh (~0.56 ms) (Dùng <i>B-Tree Index Range Scan</i>)	Chậm (~4.02 ms) (Phải <i>Full Scan</i> bằng Lua script)	MySQL thắng
Truy vấn kết nối (JOIN)	Nhanh (~0.56 ms) (Hỗ trợ <i>native</i> , tối ưu sâu)	Chậm (~1.44 ms) (Bị lỗi <i>N+1 query</i> , quét thủ công)	MySQL thắng
Composite condition	Nhanh (~0.96 ms) (<i>Query Optimizer</i> tự kết hợp)	Khá chậm (~1.68 ms) (Dùng <i>Set pre-filter + filter</i> vòng lặp)	MySQL thắng

Bảng 2.2: Kết quả thực tế đo lường so sánh hiệu năng giữa MySQL và Redis

Mặc dù phiên bản Redis tiêu chuẩn (Core Redis) bộc lộ hạn chế khi xử lý truy vấn khoảng (Range Query) và đa điều kiện (Composite Condition) do phải quét dữ liệu tuần tự (với độ phức tạp $O(N)$), hệ sinh thái Redis cung cấp một giải pháp khắc phục triệt để mang tên **RediSearch**.

Đây là một module mở rộng chính thức, biến Redis thành một công cụ tìm kiếm mạnh mẽ bằng cách tự động hóa việc tạo chỉ mục thứ cấp (Secondary Indexing) và hỗ trợ cấu trúc *Inverted Index* tối ưu.

Thay vì phải tự viết các đoạn mã (*Lua Script*) lặp qua từng phần tử một cách thủ công, RedisSearch cho phép thực hiện các truy vấn đa điều kiện và tìm kiếm toàn văn (*Full-text Search*) chỉ với một câu lệnh duy nhất.

Việc tích hợp RedisSearch trong thực tế sẽ giúp hệ thống triệt tiêu hoàn toàn nút thắt cổ chai $O(N)$, đưa tốc độ truy vấn phức tạp về mức dưới 1 mili-giây, giúp Redis vừa giữ được sức mạnh *In-memory*, vừa linh hoạt không kém gì các hệ quản trị cơ sở dữ liệu quan hệ truyền thống.

2.3 Query Processing

2.3.1 Khái niệm Query Processing

Query Processing là quá trình hệ quản trị cơ sở dữ liệu tiếp nhận yêu cầu truy vấn, phân tích cú pháp, tối ưu kế hoạch thực thi và trả về kết quả cho người dùng. Chất lượng của cơ chế xử lý truy vấn ảnh hưởng trực tiếp đến độ trễ, throughput và khả năng biểu diễn các nghiệp vụ phức tạp.

2.3.2 Cách tiếp cận của MySQL và Redis

Bảng 2.3: So sánh cách xử lý truy vấn giữa MySQL và Redis

Tiêu chí	MySQL	Redis
Ngôn ngữ truy vấn	SQL khai báo, giàu khả năng biểu diễn	Lệnh đơn giản, kết hợp Lua script để xử lý logic
Optimizer	Có query optimizer, execution plan, cost-based optimization	Không có optimizer cho truy vấn phức tạp; ứng dụng phải tự quyết định cách truy xuất
JOIN / GROUP BY	Hỗ trợ native, tối ưu nội bộ	Không hỗ trợ native; phải mô phỏng bằng nhiều lệnh hoặc Lua script
Aggregate / Sort / Limit	Hỗ trợ trực tiếp bằng SQL	Phải duyệt dữ liệu, gom nhóm và sắp xếp thủ công
Độ linh hoạt	Phù hợp truy vấn phân tích và nghiệp vụ nhiều quan hệ	Phù hợp truy cập trực tiếp theo key hoặc tập dữ liệu đã chuẩn bị sẵn

MySQL MySQL tiếp cận theo mô hình truy vấn khai báo: người dùng mô tả *cần lấy dữ liệu gì*, còn hệ quản trị tự quyết định *lấy bằng cách nào*. Với bộ tối ưu truy vấn, MySQL có thể:

- chọn index phù hợp cho điều kiện lọc;
- tối ưu thứ tự JOIN giữa nhiều bảng;
- thực hiện GROUP BY, ORDER BY, LIMIT và subquery trực tiếp;
- khi cần phân tích sâu, có thể dùng EXPLAIN ngoài phần liệt kê mã nguồn truy vấn chính.

Redis Redis không phải là hệ quản trị hướng truy vấn quan hệ. Thế mạnh của Redis là truy cập dữ liệu theo key với độ trễ rất thấp. Khi cần truy vấn phức tạp hơn, lập trình viên phải:

- thiết kế key và set index ngay từ đầu;
- tự kết hợp nhiều lệnh HGET, SMEMBERS, ZRANGE, SINTER;
- dùng Lua script để gom nhiều bước thành một thao tác phía server;
- chấp nhận rằng aggregate, sort và filtering phức tạp thường phải scan nhiều key.

2.3.3 Môi trường và dữ liệu thử nghiệm

Phần thực nghiệm sử dụng bộ dữ liệu BikeStores đã được import vào cả hai hệ:

- **MySQL:** dữ liệu nằm trong hai schema `production` và `sales`.
- **Redis:** dữ liệu được ánh xạ dưới dạng Hash cho thực thể và Set cho các tập chỉ mục.

```
PS C:\Users\HP> docker exec mysql-bike mysql -uroot -proot -e "SELECT COUNT(*) AS total_products FROM production.products; SELECT COUNT(*) AS total_orders FROM sales.orders; SELECT COUNT(*) AS total_order_items FROM sales.order_items; SELECT COUNT(*) AS total_customers FROM sales.customers;"
mysql: [Warning] Using a password on the command line interface can be insecure.
total_products
321
total_orders
1615
total_order_items
4722
total_customers
1445
PS C:\Users\HP>
```

Hình 2.20: Số bản ghi kiểm tra phía MySQL.

```
PS C:\Users\HP> docker exec redis-bike redis-cli SCARD production:products:ids
321
PS C:\Users\HP> docker exec redis-bike redis-cli SCARD sales:orders:ids
1615
PS C:\Users\HP> docker exec redis-bike redis-cli EVAL "return #redis.call('keys', 'sales:order_item:*')"
```

Hình 2.21: Số key kiểm tra phía Redis.

Bảng 2.4: Các bảng và key chính được sử dụng trong phần Query Processing

Nghệp vụ	MySQL	Redis
Sản phẩm theo danh mục	<code>production.products</code>	<code>production:product:{id}</code> <code>production:products:category:{category_id}</code>
Chi tiết đơn hàng	<code>sales.orders</code> <code>sales.order_items</code> <code>production.products</code>	<code>sales:order:{id}</code> <code>sales:order_item:{order_id}:{item_id}</code>
Doanh thu theo cửa hàng	<code>sales.stores</code> <code>sales.orders</code> <code>sales.order_items</code>	<code>sales:orders:store:{store_id}</code> Hash <code>sales:order_item:{order_id}:{item_id}</code>
Khách hàng chưa phát sinh đơn	<code>sales.customers</code> <code>sales.orders</code>	<code>sales:customers:ids</code> <code>sales:orders:customer:{customer_id}</code>

2.3.4 Thực nghiệm truy vấn

Truy vấn đơn giản trên một bảng **Mục tiêu:** lấy tất cả sản phẩm thuộc category “Mountain Bikes” (`category_id = 6`) và sắp xếp theo giá giảm dần.

MySQL

```
SELECT product_id, product_name, list_price
FROM production.products
WHERE category_id = 6
ORDER BY list_price DESC;
```

Redis

```
redis-cli EVAL "
local pids = redis.call('SMEMBERS', 'production:products:category:6')
local result = {}
for _, pid in ipairs(pids) do
    local p = redis.call('HGETALL', 'production:product:' .. pid)
    local info = {}
```



```

for i = 1, #p, 2 do info[p[i]] = p[i+1] end
table.insert(result, {tonumber(info.list_price),
    pid .. ' | ' .. info.product_name .. ' | $' .. info.list_price})
end
table.sort(result, function(a, b) return a[1] > b[1] end)
local output = {}
for _, r in ipairs(result) do table.insert(output, r[2]) end
return output
" 0

```

Nhận xét: Đây là loại truy vấn Redis xử lý khá tốt vì đã có set index theo danh mục. Tuy nhiên Redis vẫn phải tự tải từng hash và tự sắp xếp trong Lua, trong khi MySQL biểu diễn cùng nghiệp vụ ngắn gọn hơn bằng một câu SQL duy nhất.

```

mysql> SELECT product_id, product_name, list_price
-> FROM production.products
-> WHERE category_id = 6
-> ORDER BY list_price DESC;

```

product_id	product_name	list_price
43	Trek Fuel EX 9.8 27.5 Plus - 2017	5299.99
47	Trek Remedy 9.8 - 2017	5299.99
40	Trek Fuel EX 9.8 29 - 2017	4999.99
140	Trek Remedy 9.8 27.5 - 2018	4999.99
7	Trek Slash 8 27.5 - 2016	3999.99
142	Trek Fuel EX 8 29 XT - 2018	3199.99
115	Trek Fuel EX 8 29 - 2018	3199.99
139	Trek Remedy 7 27.5 - 2018	2999.99
4	Trek Fuel EX 8 29 - 2016	2899.99
137	Heller Shagawaw GX1 - 2018	2599.00
131	Heller Bloodhound Trail - 2018	2599.00
28	Surly Karate Monkey 27.5+ Frameset - 2017	2499.99
120	Trek Fuel EX 7 29 - 2018	2499.99
121	Surly Krampus Frameset - 2018	2499.99
122	Surly Troll Frameset - 2018	2499.99
42	Trek Fuel EX 5 27.5 Plus - 2017	2299.99
138	Trek Fuel EX 5 Plus - 2018	2249.99
128	Surly ECR 27.5 - 2018	1899.00
136	Trek Procaliber 6 - 2018	1799.99
8	Trek Remedy 29 Carbon Frameset - 2016	1799.99
31	Surly Wednesday - 2017	1632.99
141	Trek Stache 5 - 2018	1599.99
132	Trek Procal AL Frameset - 2018	1499.99
133	Trek Procaliber Frameset - 2018	1499.99
134	Trek Remedy 27.5 C Frameset - 2018	1499.99
135	Trek X-Caliber Frameset - 2018	1499.99
39	Trek Stache 5 - 2017	1499.99
124	Surly Krampus - 2018	1499.00
41	Haro Shift R3 - 2017	1469.99
117	Trek Ticket 5 Frame - 2018	1469.99
46	Haro SR 1.3 - 2017	1409.99
5	Heller Shagawaw Frame - 2016	1320.99
29	Trek X-Caliber 8 - 2017	999.99
30	Surly Ice Cream Truck Frameset - 2017	999.99
27	Surly Big Dummy Frameset - 2017	999.99
118	Trek X-Caliber 8 - 2018	999.99
3	Surly Wednesday Frameset - 2016	999.99
123	Trek Farley Carbon Frameset - 2018	999.99
129	Trek X-Caliber 7 - 2018	919.99
130	Trek Stache Carbon Frameset - 2018	919.99
45	Haro SR 1.2 - 2017	869.99
35	Sun Bicycles Spider 3i - 2017	832.99
36	Surly Troll Frameset - 2017	832.99
2	Ritchey Timberwolf Frameset - 2016	749.99
116	Trek Marlin 7 - 2017/2018	749.99
114	Trek Marlin 6 - 2018	579.99
38	Haro Flightline Two 26 Plus - 2017	549.99
44	Haro SR 1.1 - 2017	539.99
113	Trek Marlin 5 - 2018	489.99
127	Surly Pack Rat Frameset - 2018	469.99
126	Surly Big Fat Dummy Frameset - 2018	469.99
125	Trek Kids' Dual Sport - 2018	469.99
119	Trek Kids' Neko - 2018	469.99
34	Trek Session DH 27.5 Carbon Frameset - 2017	469.99
33	Surly Wednesday Frameset - 2017	469.99
32	Trek Farley Alloy Frameset - 2017	469.99
6	Surly Ice Cream Truck Frameset - 2016	469.99
112	Trek 820 - 2018	379.99
37	Haro Flightline One ST - 2017	379.99
1	Trek 820 - 2016	379.99

```

60 rows in set (0.02 sec)
mysql>

```

Hình 2.22: Kết quả Q1 trên MySQL.

```

" 0> > > >
1) "47 | Trek Remedy 9.8 - 2017 | 5299.99"
2) "43 | Trek Fuel EX 9.8 27.5 Plus - 2017 | 5299.99"
3) "140 | Trek Remedy 9.8 27.5 - 2018 | 4999.99"
4) "40 | Trek Fuel EX 9.8 29 - 2017 | 4999.99"
5) "7 | Trek Slash 8 27.5 - 2016 | 3999.99"
6) "115 | Trek Fuel EX 8 29 - 2018 | 3199.99"
7) "142 | Trek Fuel EX 8 29 XT - 2018 | 3199.99"
8) "139 | Trek Remedy 7 27.5 - 2018 | 2999.99"
9) "4 | Trek Fuel EX 8 29 - 2016 | 2899.99"
10) "137 | Heller Shagawaw GX1 - 2018 | 2599"
11) "131 | Heller Bloodhound Trail - 2018 | 2599"
12) "121 | Surly Krampus Frameset - 2018 | 2499.99"
13) "120 | Trek Fuel EX 7 29 - 2018 | 2499.99"
14) "28 | Surly Karate Monkey 27.5+ Frameset - 2017 | 2499.99"
15) "122 | Surly Troll Frameset - 2018 | 2499.99"
16) "42 | Trek Fuel EX 5 27.5 Plus - 2017 | 2299.99"
17) "138 | Trek Fuel EX 5 Plus - 2018 | 2249.99"
18) "128 | Surly ECR 27.5 - 2018 | 1899"
19) "136 | Trek Procaliber 6 - 2018 | 1799.99"
20) "8 | Trek Remedy 29 Carbon Frameset - 2016 | 1799.99"
21) "31 | Surly Wednesday - 2017 | 1632.99"
22) "141 | Trek Stache 5 - 2018 | 1599.99"
23) "135 | Trek X-Caliber Frameset - 2018 | 1499.99"
24) "132 | Trek Procal AL Frameset - 2018 | 1499.99"
25) "134 | Trek Remedy 27.5 C Frameset - 2018 | 1499.99"
26) "133 | Trek Procaliber Frameset - 2018 | 1499.99"
27) "39 | Trek Stache 5 - 2017 | 1499.99"
28) "124 | Surly Krampus - 2018 | 1499"
29) "117 | Trek Ticket 5 Frame - 2018 | 1469.99"
30) "41 | Haro Shift R3 - 2017 | 1469.99"
31) "46 | Haro SR 1.3 - 2017 | 1409.99"
32) "5 | Heller Shagawaw Frame - 2016 | 1320.99"
33) "123 | Trek Farley Carbon Frameset - 2018 | 999.99"
34) "30 | Surly Ice Cream Truck Frameset - 2017 | 999.99"
35) "118 | Trek X-Caliber 8 - 2018 | 999.99"
36) "29 | Trek X-Caliber 8 - 2017 | 999.99"
37) "3 | Surly Wednesday Frameset - 2016 | 999.99"
38) "27 | Surly Big Dummy Frameset - 2017 | 999.99"
39) "130 | Trek Stache Carbon Frameset - 2018 | 919.99"
40) "129 | Trek X-Caliber 7 - 2018 | 919.99"
41) "45 | Haro SR 1.2 - 2017 | 869.99"
42) "35 | Sun Bicycles Spider 3i - 2017 | 832.99"
43) "36 | Surly Troll Frameset - 2017 | 832.99"
44) "116 | Trek Marlin 7 - 2017/2018 | 749.99"
45) "2 | Ritchey Timberwolf Frameset - 2016 | 749.99"
46) "114 | Trek Marlin 6 - 2018 | 579.99"
47) "38 | Haro Flightline Two 26 Plus - 2017 | 549.99"
48) "44 | Haro SR 1.1 - 2017 | 539.99"
49) "113 | Trek Marlin 5 - 2018 | 489.99"
50) "126 | Surly Big Fat Dummy Frameset - 2018 | 469.99"
51) "6 | Surly Ice Cream Truck Frameset - 2016 | 469.99"
52) "127 | Surly Pack Rat Frameset - 2018 | 469.99"
53) "119 | Trek Kids' Neko - 2018 | 469.99"
54) "32 | Trek Farley Alloy Frameset - 2017 | 469.99"
55) "34 | Trek Session DH 27.5 Carbon Frameset - 2017 | 469.99"
56) "125 | Trek Kids' Dual Sport - 2018 | 469.99"
57) "33 | Surly Wednesday Frameset - 2017 | 469.99"
58) "112 | Trek 820 - 2018 | 379.99"
59) "37 | Haro Flightline One ST - 2017 | 379.99"
60) "1 | Trek 820 - 2016 | 379.99"
61) ""
62) "----- Q1 -----"
63) "time_start: 1774681116.452012"
64) "time_end: 1774681116.452012"
65) "elapsed_ms: 0.000"
66) "records_returned: 60"
# |

```

Hình 2.23: Kết quả Q1 trên Redis (Lua EVAL).

Truy vấn JOIN nhiều bảng Mục tiêu: lấy chi tiết đơn hàng số 1, bao gồm tên sản phẩm, số lượng, giá và tổng tiền từng dòng.

MySQL

```
SELECT o.order_id, o.order_date, p.product_name,
       oi.quantity, oi.list_price, oi.discount,
       (oi.quantity * oi.list_price * (1 - oi.discount)) AS line_total
FROM sales.orders o
JOIN sales.order_items oi ON o.order_id = oi.order_id
JOIN production.products p ON oi.product_id = p.product_id
WHERE o.order_id = 1;
```

Redis

```
redis-cli EVAL "  
local item_ids = redis.call('SMEMBERS', 'sales:order_items:order:1')  
local result = {}  
for _, iid in ipairs(item_ids) do  
    local item = redis.call('HGETALL', 'sales:order_item:1:' .. iid)  
    local info = {}  
    for i = 1, #item, 2 do info[item[i]] = item[i+1] end  
    local pname = redis.call('HGET', 'production:product:' .. info.  
        product_id, 'product_name')  
    local total = tonumber(info.quantity) * tonumber(info.list_price) *  
        (1 - tonumber(info.discount))  
    table.insert(result, pname .. ' | qty:' .. info.quantity .. ' | $' ..  
        info.list_price ..  
        ' | disc:' .. info.discount .. ' | total: ' .. string.format('%.4  
        f', total))  
end  
return result  
" 0
```

Nhận xét: MySQL thể hiện lợi thế rất rõ ở JOIN vì quan hệ giữa các bảng đã được mô hình hóa sẵn bằng khóa ngoại. Trong Redis, JOIN không tồn tại theo nghĩa native mà phải ghép nhiều key bằng tay, nên truy vấn dài hơn và khó bảo trì hơn.

```
mysql> SELECT o.order_id, o.order_date, p.product_name,
->         oi.quantity, oi.list_price, oi.discount,
->         (oi.quantity * oi.list_price * (1 - oi.discount)) AS line_total
-> FROM sales.orders o
-> JOIN sales.order_items oi ON o.order_id = oi.order_id
-> JOIN production.products p ON oi.product_id = p.product_id
-> WHERE o.order_id = 1;

order_id  order_date      product_name      quantity  list_price  discount  line_total
-----
1  2016-01-01    Trekka Tennis Original TD EX - Women's - 2016      1      599.99      0.20      479.9924
1  2016-01-01    Trek Ready29 Carbon Framest - 2016                2      1799.99      0.07      3374.9814
1  2016-01-01    Surly Straggle - 2016                              2      1569.00      0.05      2993.1000
1  2016-01-01    Trekka Tennis Original TD EX - 2016                1      599.99      0.20      479.9914
1  2016-01-01    Trek Fuel EX 8 29 - 2016                          1      2899.99      0.20      2319.9920

5 rows in set (0.03 sec)

mysql>
```

Hình 2.24: Kết quả Q2 trên MySQL.

```
# redis-cli EVAL =  
local function usec(t)  
    return tonumber(t[1]) * 1000000 + tonumber(t[2])  
end  
  
local function fmt_server_time(t)  
    return string.format('%d.%06d', tonumber(t[1]), tonumber(t[2]))  
end  
  
local t0 = redis.call('TIME')  
local time_start = fmt_server_time(t0)  
local item_ids = redis.call('SMEMBERS@', 'sales:order_items:order:1')  
local result = {}  
for _, iid in ipairs(item_ids) do  
    local item = redis.call('HGETALL', 'sales:order_item:1' .. iid)  
    local info = {}  
    for i = 1, #item, 2 do info[item[i]] = item[i+1] end  
    local pname = redis.call('HGET', 'production:product:' .. info.product_id, 'product_name')  
    local total = tonumber(info.quantity) * tonumber(info.list_price) * (1 - tonumber(info.discount))  
    table.insert(result, t.pname .. '| qty:' .. info.quantity .. '|' $'..info.list_price ..  
        '| disc:' .. info.discount .. '|' total: ' .. string.format(> '%.4f', total))  
end  
  
local n_rows = #result  
local t1 = redis.call('TIME')  
local time_end = fmt_server_time(t1)  
local elapsed_ms = (usec(t1) - usec(t0)) / 1000  
table.insert(result, '')  
table.insert(result, '----- Q2 -----')  
table.insert(result, 'time_start: ' .. time_start)  
table.insert(result, 'time_end: ' .. time_end)  
table.insert(result, 'elapsed_ms: ' .. string.gformat('%3f', elapsed_ms))  
table.insert(result, 'records_returned: ' .. n_rows)  
return result  
  
>>> >>> >>> >>> >>> >>> >>> >>> >>>  
> "Trek Remedy 29 Carbon Frameset - 2016 | qty:2 | $1799.99 | disc:0.87 | total: 3347.9814"  
> "Surly Straggler - 2016 | qty:2 | $1549.00 | disc:0.05 | total: 2943.1080"  
> "Electra Townie Original 7Q EQ - 2016 | qty:1 | $599.99 | disc:0.85 | total: 1139.9818"  
> "Trek Fuel EX 29 - 2016 | qty:1 | $2899.99 | disc:0.2 | total: 2319.9928"  
>  
> "  
> "----- Q2 -----"  
> "time_start: 1776691186.468972"  
> "time_end: 1776691186.468472"  
> "elapsed_ms: 0.000"  
>  
11> "records_returned: 5"
```

Hình 2.25: Kết quả Q2 trên Redis.

Truy vấn nhóm và tính tổng Mục tiêu: tính tổng doanh thu theo từng cửa hàng. MySQL

```
SELECT s.store_name,
       COUNT(DISTINCT o.order_id) AS total_orders,
       SUM(oi.quantity * oi.list_price * (1 - oi.discount)) AS revenue
FROM sales.stores s
JOIN sales.orders o ON s.store_id = o.store_id
JOIN sales.order_items oi ON o.order_id = oi.order_id
GROUP BY s.store_name
ORDER BY revenue DESC;
```

Redis

```
redis-cli EVAL "
local stores = redis.call('SMEMBERS', 'sales:stores:ids')
local rows = {}
for _, sid in ipairs(stores) do
    local name = redis.call('HGET', 'sales:store:' .. sid, 'store_name')
    local oids = redis.call('SMEMBERS', 'sales:orders:store:' .. sid)
    local revenue = 0
    for _, oid in ipairs(oids) do
        local iids = redis.call('SMEMBERS', 'sales:order_items:order:' ..
                                oid)
        for _, iid in ipairs(iids) do
            local key = 'sales:order_item:' .. oid .. ':' .. iid
            local q = tonumber(redis.call('HGET', key, 'quantity'))
            local p = tonumber(redis.call('HGET', key, 'list_price'))
            local d = tonumber(redis.call('HGET', key, 'discount'))
            revenue = revenue + (q * p * (1 - d))
        end
    end
    table.insert(rows, {revenue, #oids, name})
end
table.sort(rows, function(a, b) return a[1] > b[1] end)
local result = {}
for _, r in ipairs(rows) do
    table.insert(result, r[3] .. ' | Orders: ' .. r[2] ..
                ' | Revenue: ' .. string.format('%.4f', r[1]))
end
return result
" 0
```

Nhận xét: Đây là nhóm truy vấn Redis bất lợi nhất vì phải quét qua rất nhiều key rồi tự cộng dồn thủ công. MySQL xử lý tốt hơn nhờ engine tối ưu cho aggregate, grouping và join theo quan hệ.

```
mysql> SELECT s.store_name,
-> COUNT(DISTINCT o.order_id) AS total_orders,
-> SUM(o.quantity * oi.list_price * (1 - oi.discount)) AS revenue
-> FROM sales.stores s
-> JOIN sales.orders o ON s.store_id = o.store_id
-> JOIN sales.order_items oi ON o.order_id = oi.order_id
-> GROUP BY s.store_name
-> ORDER BY revenue DESC;
```

store_name	total_orders	revenue
Baldwin Bikes	1093	5215751.2775
Santa Cruz Bikes	348	1605823.0365
Rowlett Bikes	174	867542.2436

```
3 rows in set (0.06 sec)
```

```
mysql> |
```

Hình 2.26: Kết quả Q3 trên MySQL.

```
# redis-cli EVAL "
local function usec(t)
    return tonumber(t[1]) * 1000000 + tonumber(t[2])
end

local function fmt_server_time(t)
    return string.format("%d.%06d", tonumber(t[1]), tonumber(t[2]))
end

local t0 = redis.call('TIME')
local time_start = fmt_serv> er.time(t0)
local stores = redis.call('SMEMBERS', 'sales:stores:ids')
local rows = {}
for _, sid> in ipairs(stores) do
    local name = redis.call('HGET', '> sales:store:' .. sid, 'store_name')
    local oids = redis.call('SMEMBERS', 'sales:orders:store:' .. sid)
    local > revs = {}
    for _>, oid in ipairs(oids) do
        local iids = redis.call('SMEMBERS', 'sales:order_items:ord> er:' .. oid)
        for _>, iid in ipairs(iids) do
            local key = 'sales:order-item:' .. oid .. ':' .. iid
            local q = tonumber(redis.call('HGET', key, '> 'quantity'))
            local p = tonumber(redis.call('HGET', key, '> list_price'))
            local d = tonumber(redis.call('HGET', key, 'discount'))
            revenue = revenue + (q * p * (1 - d))
        end
    end
    end
    table.insert(rows, {revenue, #oids, name})
end
table.sort(rows, > function(a, b) return a[1] > b[1] end)
local result = {}
for _>, r in ipairs(rows) do
    table.insert(result, {r[1] .. ' | Orders: ' .. r[2]> .. ' .. '
        .. ' | Revenue: ' .. string.format('%> .4f', r[1])})
end
local n_rows = #result
local t1 = redis.call('T> IME')
local time_end = fmt_server.time(t1)
local > elapsed_ms = (usec(t1) - usec(t0)) / 1000
table.insert(result, '> ')
table.insert(result, '----- Q3 -----')
> table.insert(result, 'time_start: ' .. time_start)
table.insert(result, 'ti> > > > > me_end: ' .. time_end)
table.insert(res> ult, 'elapsed_ms: ' .. string.format('%> .3f', elapsed_ms))
> table.insert(result, 'records_returned: ' .. n_rows)
return> n result
" BS > > > >
1) "Baldwin Bikes | Orders: 1093 | Revenue: 5215751.2775"
2) "Santa Cruz Bikes | Orders: 348 | Revenue: 1605823.8365"
3) "Rowlett Bikes | Orders: 174 | Revenue: 867542.2436"
4) ""
5) "----- Q3 -----"
6) "time_start: 17747777009.663279"
7) "time_end: 17747777009.663279"
8) "elapsed_ms: 0.000"
9) "records_returned: 3"
```

Hình 2.27: Kết quả Q3 trên Redis.

```
mysql> EXPLAIN
-> SELECT s.store_name,
->        COUNT(DISTINCT o.order_id) AS total_orders,
->        SUM(oi.quantity * oi.list_price * (1 - oi.discount)) AS revenue
-> FROM sales.stores s
-> JOIN sales.orders o ON s.store_id = o.store_id
-> JOIN sales.order_items oi ON o.order_id = oi.order_id
-> GROUP BY s.store_name
-> ORDER BY revenue DESC;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	o	NULL	index	PRIMARY,store_id	store_id	4	NULL	1	100.00	Using index; Using temporary
1	SIMPLE	oi	NULL	ref	PRIMARY	PRIMARY	4	sales.o.order_id	1	100.00	NULL
1	SIMPLE	s	NULL	eq_ref	PRIMARY	PRIMARY	4	sales.o.store_id	1	100.00	NULL

```
3 rows in set, 1 warning (0.04 sec)

mysql>
```

Hình 2.28: Execution plan (EXPLAIN) cho Q3 trên MySQL — minh họa cách optimizer ước lượng chi phí.

Truy vấn Top-N Mục tiêu: tìm 5 sản phẩm bán chạy nhất theo tổng số lượng bán ra.

```
SELECT p.product_id, p.product_name, SUM(oi.quantity) AS total_sold
FROM sales.order_items oi
JOIN production.products p ON oi.product_id = p.product_id
GROUP BY p.product_id
ORDER BY total_sold DESC
LIMIT 5;
```

Ghi chú: BikeStores có nhiều `product_id` trùng `product_name`; chỉ GROUP BY `p.product_id` (không gom theo tên) để khớp Redis.

Redis

```
redis-cli EVAL "  
local pids = redis.call('SMEMBERS', 'production:products:ids')  
local sales = {}  
for _, pid in ipairs(pids) do sales[pid] = 0 end  
local oids = redis.call('SMEMBERS', 'sales:orders:ids')  
for _, oid in ipairs(oids) do  
    local iids = redis.call('SMEMBERS', 'sales:order_items:order:' .. oid  
    )  
    for _, iid in ipairs(iids) do  
        local key = 'sales:order_item:' .. oid .. ':' .. iid  
        local pid = redis.call('HGET', key, 'product_id')  
        local qty = tonumber(redis.call('HGET', key, 'quantity'))  
        if sales[pid] then sales[pid] = sales[pid] + qty end  
    end  
end  
local sorted = {}  
for pid, qty in pairs(sales) do  
    if qty > 0 then  
        local name = redis.call('HGET', 'production:product:' .. pid, '  
        product_name')  
        table.insert(sorted, {qty, name, pid})  
    end  
end  
table.sort(sorted, function(a, b) return a[1] > b[1] end)  
local result = {}  
for i = 1, math.min(5, #sorted) do  
    table.insert(result, '#' .. i .. ' | id ' .. sorted[i][3] .. ' | ' ..  
        sorted[i][2] .. ' | Sold: ' .. sorted[i][1])  
end  
return result  
" 0
```

Nhận xét: Top-N là dạng truy vấn tiêu biểu cho sự khác biệt giữa hai hệ. Với MySQL, đây là truy vấn tổng hợp tiêu chuẩn. Với Redis, nếu không thiết kế sẵn sorted set hoặc materialized leaderboard thì phải quét toàn bộ dữ liệu rồi sắp xếp thủ công, gây tốn CPU và thời gian.

```
mysql> SELECT p.product_id, p.product_name, SUM(oi.quantity) AS total_sold
-> FROM sales.order_items oi
-> JOIN production.products p ON oi.product_id = p.product_id
-> GROUP BY p.product_id
-> ORDER BY total_sold DESC
-> LIMIT 5;
```

product_id	product_name	total_sold
6	Surly Ice Cream Truck Frameset - 2016	167
13	Electra Cruiser 1 (24-Inch) - 2016	157
16	Electra Townie Original 7D EQ - 2016	156
23	Electra Girl's Hawaii 1 (20-inch) - 2015/2016	154
7	Trek Slash 8 27.5 - 2016	154

5 rows in set (0.11 sec)

Hình 2.29: Kết quả Q4 trên MySQL.

```
# redis-cli EVAL "
local function usec(t)
return tonumber(t[1]) * 1000000 + tonumber(t[2])
end
local function fat_server_time(t)
return string.format("%d.%06d", tonumber(t[1]), tonumber(t[2]))
end
local qb = redis.call("TIME")
local time_start = fat_server_time(qb)
local pids = redis.call("SMEMBERS", "production:products:ids")
local sales = {}
for _, pid in ipairs(pids) do sales[pid] = 0 end
local oids = redis.call("SMEMBERS", "sales:orders:ids")
for _, oid in ipairs(oids) do
local aids = redis.call("SMEMBERS", "sales:order-items:order:" .. oid)
for _, aid in ipairs(aids) do
local key = "sales:orders:" .. aid .. ":" .. pid
local val = redis.call("HGET", key, "product_id")
local qty = tonumber(redis.call("HGET", key, "quantity"))
if sales[pid] then sales[pid] = sales[pid] + qty end
end
end
end
local sorted = {}
for pid, qty in pairs(sales) do
if qty > 0 then
local name = redis.call("HGET", "production:product:" .. pid, "product_name")
table.insert(sorted, {qty, name, pid})
end
end
table.sort(sorted, function(a, b) return a[1] > b[1] end)
local result = {}
for i = 1, math.min(5, #sorted) do
table.insert(result, {sorted[i][2], sorted[i][3], sorted[i][1]})
end
local n_rows = #result
local t2 = redis.call("TIME")
local time_end = fat_server_time(t2)
local elapsed_us = (usec(t2) - usec(t1)) / 1000
table.insert(result, {elapsed_us})
table.insert(result, {n_rows})
table.insert(result, {time_start, time_end})
table.insert(result, {elapsed_us, string.format("%.3f", elapsed_us) .. "d_us"})
table.insert(result, {records_returned: n_rows})
return result
" 0 > >
```

```
1) 1) Surly Ice Cream Truck Frameset - 2016 | Sold: 167
2) 2) Electra Cruiser 1 (24-Inch) - 2016 | Sold: 157
3) 3) Electra Townie Original 7D EQ - 2016 | Sold: 156
4) 4) Electra Girl's Hawaii 1 (20-inch) - 2015/2016 | Sold: 154
5) 5) Trek Slash 8 27.5 - 2016 | Sold: 154
6) 6)
7) 7)
8) 8)
9) 9)
10) 10)
11) 11)
```

Hình 2.30: Kết quả Q4 trên Redis.

```
mysql> EXPLAIN
-> SELECT p.product_id, p.product_name, SUM(oi.quantity) AS total_sold
-> FROM sales.order_items oi
-> JOIN production.products p ON oi.product_id = p.product_id
-> GROUP BY p.product_id
-> ORDER BY total_sold DESC
-> LIMIT 5;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	oi	NULL	ALL	product_id	NULL	NULL	NULL	1	100.00	Using temporary; Using filesort
1	SIMPLE	p	NULL	eq_ref	PRIMARY,category_id,brand_id	PRIMARY	4	sales.oi.product_id	1	100.00	NULL

2 rows in set, 1 warning (0.03 sec)

Hình 2.31: Execution plan (EXPLAIN) cho Q4 trên MySQL.

Truy vấn lồng nhau / anti-join Mục tiêu: tìm các khách hàng chưa từng phát sinh đơn hàng.

```
SELECT c.customer_id, c.first_name, c.last_name
FROM sales.customers c
WHERE c.customer_id NOT IN (
  SELECT DISTINCT customer_id
  FROM sales.orders
  WHERE customer_id IS NOT NULL
);
```

Redis

```
redis-cli EVAL "
local cids = redis.call("SMEMBERS", "sales:customers:ids")
local result = {}
for _, cid in ipairs(cids) do
local oids = redis.call("SMEMBERS", "sales:orders:customer:" .. cid)
if #oids == 0 then
local fname = redis.call("HGET", "sales:customer:" .. cid, "
first_name")
```

```

        local lname = redis.call('HGET', 'sales:customer:' .. cid, '
            last_name')
        table.insert(result, cid .. ' | ' .. fname .. ' ' .. lname)
    end
end
return result
" 0

```

Nhận xét: MySQL hỗ trợ tự nhiên cho dạng anti-join hoặc subquery. Redis chỉ có thể đạt cùng kết quả nếu ta thiết kế sẵn set đơn hàng theo từng khách hàng; nếu thiếu cấu trúc đó thì chi phí truy vấn sẽ tăng rất mạnh. Với bộ BikeStores mặc định, danh sách khách chưa có đơn có thể rỗng.

```
mysql> SELECT c.customer_id, c.first_name, c.last_name
-> FROM sales.customers c
-> WHERE c.customer_id NOT IN (
-> SELECT DISTINCT customer_id
-> FROM sales.orders
-> WHERE customer_id IS NOT NULL
-> );
Empty set (0.03 sec)

mysql> |
```

Hình 2.32: Kết quả Q5 trên MySQL.

[illegible]

Hình 2.33: Kết quả Q5 trên Redis.

```
mysql> EXPLAIN
-> SELECT c.customer_id, c.first_name, c.last_name
-> FROM sales.customers c
-> WHERE c.customer_id NOT IN (
->     SELECT DISTINCT customer_id
->     FROM sales.orders
->     WHERE customer_id IS NOT NULL
-> );
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	c	NONE	ALL	NONE	NONE	NONE	NONE	1445	100.00	Using where
2	SUBQUERY	orders	NONE	index	customer_id	customer_id	5	NONE	1	100.00	Using where; Using index

```
2 rows in set, 1 warning (0.02 sec)

mysql>
```

Hình 2.34: Execution plan (EXPLAIN) cho Q5 trên MySQL.

2.3.5 Phân tích sâu hơn về khả năng biểu diễn truy vấn

Độ phức tạp triển khai

- Với MySQL, phần lớn truy vấn nghiệp vụ chỉ cần viết SQL đúng logic.
- Với Redis, hiệu quả phụ thuộc mạnh vào thiết kế key, set index và việc chấp nhận duplicate data để phục vụ truy vấn.
- Điều này cho thấy Redis thường dịch chuyển chi phí từ “lúc truy vấn” sang “lúc thiết kế mô hình dữ liệu”.

Khả năng mở rộng truy vấn

- Khi yêu cầu nghiệp vụ thay đổi, MySQL thường chỉ cần sửa câu SQL hoặc thêm index.
- Trong Redis, thay đổi nghiệp vụ có thể buộc phải tạo thêm key pattern, thêm set index hoặc đồng bộ lại dữ liệu.
- Vì vậy Redis phù hợp hơn với các truy vấn truy cập nhanh, cấu trúc đã biết trước và lặp lại nhiều lần.

2.3.6 Tổng hợp ưu và nhược điểm

Bảng 2.5: Tổng hợp ưu và nhược điểm của MySQL và Redis trong Query Processing

	MySQL	Redis
Ưu điểm	Hỗ trợ SQL đầy đủ, JOIN, GROUP BY, ORDER BY, subquery và optimizer mạnh.	Truy cập theo key cực nhanh, latency thấp, rất hiệu quả cho truy vấn đơn giản hoặc dữ liệu đã được tiền xử lý.
Nhược điểm	Không nhanh bằng Redis trong các truy cập key-value cực đơn giản.	Thiếu ngôn ngữ truy vấn quan hệ, khó bảo trì khi số lượng truy vấn nghiệp vụ tăng, aggregate và sorting phức tạp thường chậm.

2.3.7 Kết luận

Từ các truy vấn đã phân tích, có thể thấy MySQL phù hợp hơn cho các hệ thống nghiệp vụ cần truy vấn linh hoạt, nhiều bảng quan hệ, tổng hợp dữ liệu và thống kê. Redis phù hợp hơn khi yêu cầu chính là truy xuất cực nhanh theo key, đọc dữ liệu đã được chuẩn bị trước hoặc dùng làm lớp cache cho các kết quả truy vấn nặng.

Nếu hệ thống cần cả hai mục tiêu, hướng triển khai hợp lý là:

- dùng MySQL làm nguồn dữ liệu chính cho các truy vấn nghiệp vụ và báo cáo;
- dùng Redis để cache kết quả các truy vấn phổ biến hoặc lưu các leaderboard, counters, session;
- chỉ nên mô phỏng truy vấn phức tạp trên Redis khi mô hình key đã được thiết kế riêng để phục vụ đúng truy vấn đó.

2.4 Transaction

2.4.1 Mô hình ACID trong MySQL và Redis

ACID (Atomicity, Consistency, Isolation, Durability) là tập hợp bốn thuộc tính cốt lõi đảm bảo tính đúng đắn và toàn vẹn của transaction trong hệ quản trị cơ sở dữ liệu. Một hệ thống được coi là tuân thủ ACID khi đảm bảo đầy đủ cả bốn tính chất này trong mọi tình huống, kể cả khi xảy ra lỗi hoặc sự cố hệ thống.

Atomicity (Tính nguyên tử) Atomicity đảm bảo rằng một transaction được thực thi như một đơn vị không thể chia nhỏ: hoặc toàn bộ các thao tác được thực hiện thành công, hoặc không có thao tác nào được áp dụng.

Trong MySQL, engine InnoDB cung cấp cơ chế COMMIT và ROLLBACK để đảm bảo tính nguyên tử. Các thay đổi trong transaction được ghi nhận thông qua undo log, cho phép hệ thống hoàn tác toàn bộ khi xảy ra lỗi. Theo tài liệu chính thức, InnoDB hỗ trợ transaction với khả năng commit, rollback và phục hồi sau crash nhằm bảo vệ dữ liệu người dùng.

Ngược lại, Redis cung cấp cơ chế transaction thông qua MULTI và EXEC. Redis đảm bảo rằng các lệnh trong transaction được thực thi tuần tự và không bị xen kẽ bởi các client khác. Tuy nhiên, Redis không hỗ trợ rollback; nếu một lệnh trong transaction thất bại, các lệnh trước đó vẫn giữ nguyên. Điều này cho thấy Redis chỉ đảm bảo atomicity ở mức thực thi tuần tự, không phải atomicity theo nghĩa ACID đầy đủ.

Consistency (Tính nhất quán) Consistency đảm bảo rằng trạng thái của hệ thống luôn chuyển từ một trạng thái hợp lệ sang một trạng thái hợp lệ khác sau khi transaction hoàn tất.

MySQL đảm bảo consistency thông qua các ràng buộc dữ liệu như FOREIGN KEY, UNIQUE, và các quy tắc toàn vẹn. Các ràng buộc này được enforced trực tiếp bởi hệ quản trị cơ sở dữ liệu, giúp ngăn chặn dữ liệu không hợp lệ.

Trong khi đó, Redis không cung cấp cơ chế constraint ở mức hệ thống. Việc đảm bảo consistency phụ thuộc hoàn toàn vào logic của application. Điều này làm tăng tính linh hoạt nhưng đồng thời đặt trách nhiệm đảm bảo tính đúng đắn dữ liệu lên phía lập trình viên.

Isolation (Tính cô lập) Isolation đảm bảo rằng các transaction đồng thời không ảnh hưởng lẫn nhau và kết quả thực thi tương đương với việc thực hiện tuần tự.

MySQL hỗ trợ đầy đủ bốn mức isolation theo chuẩn SQL: Read Uncommitted, Read Committed, Repeatable Read và Serializable. Các mức này được triển khai thông qua cơ chế locking và multi-version concurrency control (MVCC), cho phép kiểm soát sự tương tác giữa các transaction.

Redis, do sử dụng mô hình single-threaded, đảm bảo rằng các lệnh được thực thi tuần tự. Theo tài liệu chính thức, trong một transaction Redis, “các lệnh được tuần tự hóa và thực thi liên tục, không bị gián đoạn bởi client khác”. Điều này giúp Redis tránh được race condition ở mức command, tuy nhiên không cung cấp isolation theo nghĩa transaction đầy đủ.

Durability (Tính bền vững) Durability đảm bảo rằng sau khi một transaction được commit, các thay đổi sẽ được lưu trữ vĩnh viễn, ngay cả khi hệ thống gặp sự cố.

MySQL sử dụng cơ chế Write-Ahead Logging (WAL) và redo log để đảm bảo dữ liệu được ghi xuống disk trước khi commit hoàn tất. Điều này cho phép hệ thống phục hồi trạng thái dữ liệu sau crash.

Redis cung cấp hai cơ chế persistence chính: RDB (snapshot) và AOF (append-only file). Theo tài liệu Redis, khi sử dụng AOF, các transaction được ghi xuống disk như một thao tác duy nhất, tuy

nhien trong trường hợp crash, có thể xảy ra ghi không đầy đủ và cần phục hồi bằng công cụ chuyên dụng. Do đó, durability trong Redis phụ thuộc vào cấu hình và có thể đánh đổi với hiệu năng.

Bảng 2.6: So sánh mô hình ACID giữa MySQL (InnoDB) và Redis

Thuộc tính	MySQL (InnoDB)	Redis
Atomicity	Đảm bảo đầy đủ thông qua COMMIT và ROLLBACK; sử dụng undo log để hoàn tác toàn bộ transaction khi xảy ra lỗi.	Chỉ đảm bảo thực thi tuần tự thông qua MULTI/EXEC hoặc Lua script; không hỗ trợ rollback, do đó không đảm bảo atomicity theo nghĩa ACID đầy đủ.
Consistency	Được đảm bảo bởi các ràng buộc dữ liệu như FOREIGN KEY, UNIQUE và các quy tắc toàn vẹn, enforced trực tiếp bởi DBMS.	Không có cơ chế constraint ở mức hệ thống; tính nhất quán phụ thuộc vào logic của application.
Isolation	Hỗ trợ đầy đủ các mức isolation (Read Uncommitted, Read Committed, Repeatable Read, Serializable) thông qua locking và MVCC.	Đảm bảo thực thi tuần tự nhờ mô hình single-threaded; không hỗ trợ isolation theo nghĩa transaction đầy đủ.
Durability	Đảm bảo thông qua Write-Ahead Logging (WAL) và redo log; dữ liệu được phục hồi sau crash.	Phụ thuộc vào cơ chế persistence (RDB hoặc AOF); durability có thể đánh đổi với hiệu năng và không đảm bảo tuyệt đối trong mọi cấu hình.

Đánh giá tổng hợp Từ các phân tích trên, có thể thấy rằng MySQL (InnoDB) là một hệ quản trị cơ sở dữ liệu tuân thủ đầy đủ mô hình ACID, với các cơ chế được tích hợp sâu trong hệ thống để đảm bảo tính toàn vẹn dữ liệu.

Ngược lại, Redis không phải là một hệ ACID-compliant theo nghĩa truyền thống. Redis chỉ cung cấp một số đảm bảo như thực thi tuần tự và atomicity ở mức nhóm lệnh, trong khi các yếu tố như rollback và constraint không được hỗ trợ trực tiếp.

Sự khác biệt này phản ánh hai triết lý thiết kế:

- MySQL: ưu tiên tính toàn vẹn và nhất quán dữ liệu (strong consistency)
- Redis: ưu tiên hiệu năng và độ trễ thấp (high performance, eventual correctness)

2.4.2 Hành vi Transaction trong các kịch bản thực tế

Để đánh giá cụ thể sự khác biệt trong cơ chế transaction giữa MySQL và Redis, chúng tôi tiến hành triển khai một kịch bản thực tế với đầy đủ các bước xử lý nghiệp vụ.

Kịch bản: Tạo đơn hàng Kịch bản bao gồm ba bước chính:

- Tạo đơn hàng mới
- Thêm chi tiết đơn hàng
- Cập nhật tồn kho

Yêu cầu hệ thống là đảm bảo tính toàn vẹn: toàn bộ các thao tác phải thành công hoặc bị hủy bỏ hoàn toàn.

Triển khai trên MySQL Kịch bản thành công

```

1 START TRANSACTION;
2
3 INSERT INTO sales.orders(customer_id, order_status, order_date,
4   required_date, shipped_date, store_id, staff_id)
5 VALUES(1, 1, '2026-03-02', '2026-03-10', NULL, 1, 1);
6
7 SET @new_order_id = LAST_INSERT_ID();
8
9 INSERT INTO sales.order_items(order_id, item_id, product_id, quantity,
10  list_price, discount)
11 VALUES(@new_order_id, 1, 1, 2, 379.99, 0.10);
12
13 UPDATE production.stocks
14 SET quantity = quantity - 2
15 WHERE store_id = 1 AND product_id = 1;
16
17 COMMIT;

```

Dữ liệu trước khi thực thi transaction

	order_id	customer_id	order_status	order_date	required_date	shipped_date	store_id	staff_id
▶	1617	1	1	2026-03-02	2026-03-10	NULL	1	1
	1616	1	1	2026-03-02	2026-03-10	NULL	1	1
	1615	136	3	2018-12-28	2018-12-28	NULL	3	8
	1614	135	3	2018-11-28	2018-11-28	NULL	3	8
	1613	1	3	2018-11-18	2018-11-18	NULL	2	6
	1612	3	3	2018-10-21	2018-10-21	NULL	1	3
	1611	6	3	2018-09-06	2018-09-06	NULL	2	7
	1610	15	3	2018-08-25	2018-08-25	NULL	2	7
	1609	10	3	2018-08-23	2018-08-23	NULL	2	7
	1608	53	3	2018-07-12	2018-07-12	NULL	1	2
	1607	33	3	2018-07-11	2018-07-11	NULL	1	2

Hình 2.35: Bảng orders trước khi transaction

	order_id	item_id	product_id	quantity	list_price	discount
▶	1617	1	1	2	379.99	0.10
	1616	1	1	2	379.99	0.10
	1615	3	182	1	2499.99	0.20
	1615	2	214	1	899.99	0.07
	1615	1	197	2	2299.99	0.20
	1614	3	213	2	269.99	0.20
	1614	2	159	2	2299.99	0.07
	1614	1	124	1	1499.00	0.07
	1613	2	283	2	319.99	0.05
	1613	1	153	1	4999.99	0.07
	1612	5	50	1	1550.00	0.10

orders 9 order_items 12 ×

Hình 2.36: Bảng order_items trước khi transaction

	quantity
▶	23

Hình 2.37: Bảng stocks có product_id = 1 và store_id = 1 trước khi transaction

Dữ liệu sau khi thực thi transaction

#	Time	Action	Message	Duration / Fetch
✓ 1	09:41:27	START TRANSACTION	0 row(s) affected	0.000 sec
✓ 2	09:41:27	INSERT INTO sales.orders(customer_id, order_status, order_date, required_date, shipped_date,...	1 row(s) affected	0.000 sec
✓ 3	09:41:27	SET @new_order_id = LAST_INSERT_ID()	0 row(s) affected	0.000 sec
✓ 4	09:41:27	INSERT INTO sales.order_items(order_id, item_id, product_id, quantity, list_price, discount) VAL...	1 row(s) affected	0.016 sec
✓ 5	09:41:27	UPDATE production.stocks SET quantity = quantity - 2 WHERE store_id = 1 AND product_id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
✓ 6	09:41:27	COMMIT	0 row(s) affected	0.016 sec

Hình 2.38: Thông báo lệnh thực thi thành công

	order_id	customer_id	order_status	order_date	required_date	shipped_date	store_id	staff_id
▶	1618	1	1	2026-03-02	2026-03-10	NULL	1	1
	1617	1	1	2026-03-02	2026-03-10	NULL	1	1
	1616	1	1	2026-03-02	2026-03-10	NULL	1	1
	1615	136	3	2018-12-28	2018-12-28	NULL	3	8
	1614	135	3	2018-11-28	2018-11-28	NULL	3	8
	1613	1	3	2018-11-18	2018-11-18	NULL	2	6
	1612	3	3	2018-10-21	2018-10-21	NULL	1	3
	1611	6	3	2018-09-06	2018-09-06	NULL	2	7
	1610	15	3	2018-08-25	2018-08-25	NULL	2	7
	1609	10	3	2018-08-23	2018-08-23	NULL	2	7
	1608	53	3	2018-07-12	2018-07-12	NULL	1	2

Hình 2.39: Bảng orders sau khi transaction

	order_id	item_id	product_id	quantity	list_price	discount
▶	1618	1	1	2	379.99	0.10
	1617	1	1	2	379.99	0.10
	1616	1	1	2	379.99	0.10
	1615	3	182	1	2499.99	0.20
	1615	2	214	1	899.99	0.07
	1615	1	197	2	2299.99	0.20
	1614	3	213	2	269.99	0.20
	1614	2	159	2	2299.99	0.07
	1614	1	124	1	1499.00	0.07
	1613	2	283	2	319.99	0.05
	1613	1	153	1	4999.99	0.07

Hình 2.40: Bảng order_items sau khi transaction

	quantity
▶	21

Hình 2.41: Bảng stocks có product_id = 1 và store_id = 1 sau khi transaction

Phân tích:

Kết quả thực nghiệm cho thấy transaction đã được thực thi thành công, khi toàn bộ các thao tác bao gồm thêm bản ghi vào bảng `orders`, thêm dữ liệu vào bảng `order_items` và cập nhật tồn kho trong bảng `stocks` đều được áp dụng.

Cụ thể, bản ghi mới xuất hiện trong bảng `orders` với `order_id` tương ứng, đồng thời các bản ghi liên quan trong bảng `order_items` cũng được tạo ra với khóa ngoại trở về đơn hàng này. Bên cạnh đó, giá trị tồn kho trong bảng `production.stocks` đã giảm đúng theo số lượng đặt hàng, phản ánh việc cập nhật dữ liệu được thực hiện chính xác.

Kết quả này chứng minh rằng MySQL đảm bảo tính **Atomicity**, khi toàn bộ các thao tác trong transaction được thực hiện như một đơn vị duy nhất và chỉ được ghi nhận khi tất cả đều thành công. Không tồn tại trạng thái trung gian trong đó chỉ một phần dữ liệu được cập nhật.

Ngoài ra, tính **Consistency** cũng được duy trì, khi các ràng buộc khóa ngoại giữa các bảng (`orders`, `order_items`, `products`, `stocks`) vẫn được đảm bảo. Điều này cho thấy dữ liệu sau transaction vẫn nằm trong trạng thái hợp lệ theo các quy tắc toàn vẹn đã định nghĩa.

Bên cạnh đó, cơ chế transaction của InnoDB còn đảm bảo **Isolation**, khi các thao tác được thực thi mà không bị ảnh hưởng bởi các transaction đồng thời, và **Durability**, khi các thay đổi được ghi nhận vĩnh viễn sau khi `COMMIT`.

Nhận xét:

Kịch bản này minh họa rõ ràng cách MySQL thực thi transaction theo mô hình ACID, trong đó database đóng vai trò trung tâm trong việc đảm bảo tính toàn vẹn và nhất quán dữ liệu, mà không yêu cầu logic xử lý phức tạp từ phía application.

Kịch bản thất bại (Rollback)

Mô tả:

Trong kịch bản này, transaction được thực hiện với điều kiện tồn kho không đủ để đáp ứng số lượng đặt hàng. Khi thao tác cập nhật tồn kho dẫn đến giá trị âm, transaction được kỳ vọng sẽ bị hủy bỏ nhằm đảm bảo tính toàn vẹn dữ liệu.

Thực thi:

```

1 DELIMITER //
2
3 CREATE PROCEDURE test_tx()
4 BEGIN
5     DECLARE v_stock INT;
6
7     START TRANSACTION;
8
9     UPDATE production.stocks
10    SET quantity = quantity - 5
11    WHERE store_id = 1 AND product_id = 6;
12
13    SELECT quantity INTO v_stock
14    FROM production.stocks
15    WHERE store_id = 1 AND product_id = 6;
16
17    IF v_stock < 0 THEN
18        ROLLBACK;
19    ELSE
20        COMMIT;
21    END IF;
22
23 END //
24

```

```
25 DELIMITER ;
26 CALL test_tx();
```

Dữ liệu trước khi thực thi:

	quantity
▶	0

Hình 2.42: Bảng stocks có product_id = 6 và store_id = 1 trước khi transaction

Dữ liệu sau khi thực thi:

#	Time	Action	Message	Duration / Fetch
1	11:21:12	CALL test_tx()	0 row(s) affected	0.000 sec

Hình 2.43: Kết quả sau khi thực thi procedure test_tx

Result Grid				Filter Rows:
	quantity			
	0			

Hình 2.44: Bảng stocks có product_id = 6 và store_id = 1 sau khi transaction

Phân tích:

Kết quả thực nghiệm cho thấy khi một điều kiện không hợp lệ xảy ra trong transaction (cụ thể là tồn kho không đủ), toàn bộ transaction đã bị rollback. Điều này đảm bảo rằng không có bất kỳ thay đổi nào được ghi nhận trong hệ thống, kể cả những thao tác đã thực hiện trước đó trong transaction.

Cơ chế này phản ánh rõ tính **Atomicity**, khi transaction được xem như một đơn vị không thể chia nhỏ: hoặc tất cả các thao tác đều thành công, hoặc không có thao tác nào được áp dụng. Trong trường hợp này, việc cập nhật tồn kho không hợp lệ đã khiến toàn bộ transaction bị hủy bỏ.

Ngoài ra, tính **Consistency** cũng được đảm bảo, khi hệ thống không rơi vào trạng thái dữ liệu không hợp lệ (ví dụ tồn kho âm hoặc tồn tại order không hợp lệ). Các ràng buộc toàn vẹn dữ liệu vẫn được duy trì sau khi rollback.

Cơ chế rollback của InnoDB dựa trên undo log, cho phép khôi phục trạng thái dữ liệu trước khi transaction bắt đầu. Điều này đặc biệt quan trọng trong các hệ thống thực tế, nơi các thao tác thường phụ thuộc lẫn nhau và cần được thực hiện một cách toàn vẹn.

Nhận xét:

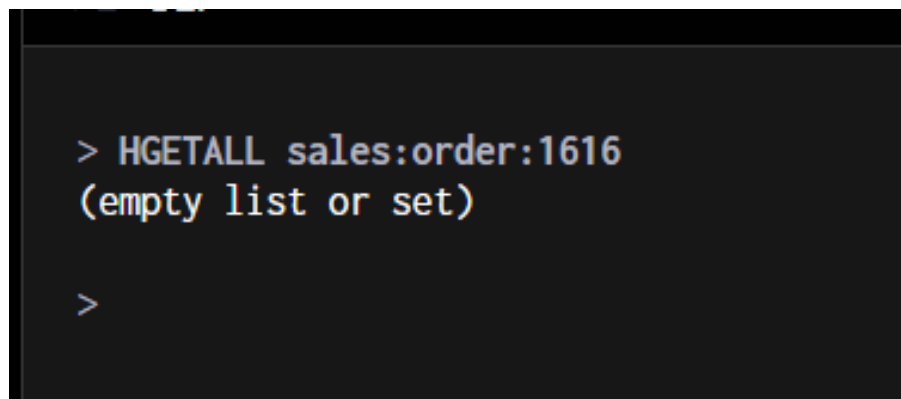
Kịch bản thất bại minh họa vai trò quan trọng của transaction trong việc bảo vệ dữ liệu khỏi các trạng thái không nhất quán. Khả năng rollback là một trong những yếu tố cốt lõi giúp MySQL tuân thủ đầy đủ mô hình ACID, đặc biệt trong các nghiệp vụ phức tạp như quản lý đơn hàng và tồn kho.

Triển khai trên Redis Kịch bản thành công

```

1 MULTI
2 INCR sales:orders:next_id
3 HSET sales:order:1616 customer_id 1 order_status 1 order_date "2026-03-02"
  " required_date "2026-03-10" shipped_date "" store_id 1 staff_id 1
4 SADD sales:orders:ids 1616
5 SADD sales:orders:customer:1 1616
6 SADD sales:orders:store:1 1616
7 HSET sales:order_item:1616:1 product_id 1 quantity 2 list_price 379.99
  discount 0.10
8 SADD sales:order_items:order:1616 1
9 HINCRBY production:stock:1:1 quantity -2
10 EXEC
  
```

Trước khi thực thi: Trong bộ dữ liệu được cung cấp, order có 1615 rows. Vì vậy order_id 1616 chưa tồn tại trong hệ thống.



Hình 2.45: Kết quả tìm kiếm order có id 1616


```
> HGETALL production:stock:1:1
1) "quantity"
2) "27"

>
```

Hình 2.46: Kết quả tìm kiếm stock có product_id = 1 và store_id = 1 trong redis trước khi thực thi transaction

Sau khi thực thi:

```
> EXEC
1) "1"
2) "7"
3) "1"
4) "1"
5) "1"
6) "4"
7) "1"
8) "25"
```

Hình 2.47: Sau khi chạy tổ hợp các câu lệnh trên

```
> HGETALL production:stock:1:1
1) "quantity"
2) "25"
```

Hình 2.48: Kết quả tìm kiếm stock có product_id = 1 và store_id = 1 trong redis sau khi thực thi thành công

```
> HGETALL sales:order:1616
1) "customer_id"
2) "1"
3) "order_status"
4) "1"
5) "order_date"
6) "2026-03-02"
7) "required_date"
8) "2026-03-10"
9) "shipped_date"
10) ""
11) "store_id"
12) "1"
13) "staff_id"
14) "1"

>
```

Hình 2.49: Kết quả tìm kiếm order có id = 1616 sau khi thực thi thành công

Bên cạnh đó, ta có thể dùng Lua script để gom hết các lệnh vào 1 block nguyên tử

```
1 EVAL "
2 local oid = 1617
3 redis.call('HSET', 'sales:order:' .. oid,
4   'customer_id', '1', 'order_status', '1',
5   'order_date', '2026-03-02', 'required_date', '2026-03-10',
6   'shipped_date', '', 'store_id', '1', 'staff_id', '1')
7 redis.call('SADD', 'sales:orders:ids', oid)
8 redis.call('HINCRBY', 'production:stock:1:1', 'quantity', -2)
9 return 'OK'
10 " 0
```

```
> EVAL "
local oid = 1617 redis.call('HSET', 'sales:order:' .. oid,
'customer_id', '1', 'order_status', '1',
'order_date', '2026-03-02', 'required_date', '2026-03-10',
'shipped_date', '', 'store_id', '1', 'staff_id', '1')
redis.call('SADD', 'sales:orders:ids', oid)
redis.call('HINCRBY', 'production:stock:1:1', 'quantity', -2)
return 'OK'
" 0
"OK"

> HGETALL sales:order:1617
1) "customer_id"
2) "1"
3) "order_status"
4) "1"
5) "order_date"
6) "2026-03-02"
7) "required_date"
8) "2026-03-10"
9) "shipped_date"
10) ""
11) "store_id"
12) "1"
13) "staff_id"
14) "1"

>
```

Hình 2.50: Kết quả sau khi thực thi Lua script thành công

Kịch bản thất bại

```
1 EVAL "
2 local stock = tonumber(redis.call('HGET', 'production:stock:1:6', '
   quantity'))
3 if stock < 5 then
4     return 'FAILED: Not enough stock (have ' .. stock .. ', need 5)'
5 end
6 redis.call('HINCRBY', 'production:stock:1:6', 'quantity', -5)
7 return 'OK: Stock reduced to ' .. (stock - 5)
8 " 0
```

```
> HGET production:stock:1:6 quantity
"0"

>
```

Hình 2.51: Quantity của stock có product_id = 6 và store_id = 1 trước khi thực thi

```
> EVAL " local stock = tonumber(redis.call('HGET', 'production:stock:1:6', 'quantity')) if stock < 5 then return 'FAILED: Not enough stock (have ' .. stock .. ', need 5)' end redis.call('HINCRBY', 'p
roduction:stock:1:6', 'quantity', -5) return 'OK: Stock reduced to ' .. (stock - 5) * 0
'FAILED: Not enough stock (have 0, need 5)'"
> HGET production:stock:1:6 quantity
"0"
```

Hình 2.52: Quantity của stock có product_id = 6 và store_id = 1 sau khi thực thi

Phân tích:

Redis đảm bảo rằng các lệnh trong khối MULTI/EXEC được thực thi tuần tự và không bị xen kẽ bởi các client khác, từ đó duy trì tính nhất quán trong quá trình thực thi. Tuy nhiên, cơ chế này không cung cấp khả năng rollback. Khi xảy ra lỗi logic hoặc điều kiện không hợp lệ, các lệnh đã được thực thi vẫn không thể bị hoàn tác, dẫn đến nguy cơ dữ liệu bị thay đổi không mong muốn.

Việc sử dụng Lua script cho phép đóng gói toàn bộ logic kiểm tra và cập nhật trong một khối thực thi nguyên tử duy nhất. Điều này giúp ngăn chặn các thay đổi không hợp lệ bằng cách dừng thực thi trước khi dữ liệu bị sửa đổi. Tuy nhiên, cách tiếp cận này mang tính chất *phòng ngừa* (preventive) hơn là *khôi phục* (recovery), và do đó không thể thay thế hoàn toàn cơ chế transaction với rollback như trong MySQL.

Cơ chế WATCH (Optimistic Locking) trong Redis

Redis cung cấp lệnh WATCH như một cơ chế *optimistic locking* để phát hiện xung đột khi nhiều client cùng thao tác trên một key. Khi một key được theo dõi, Redis sẽ kiểm tra xem key đó có bị thay đổi trước khi EXEC được gọi hay không.

Nếu key bị thay đổi bởi client khác, toàn bộ transaction sẽ bị hủy và EXEC trả về nil; ngược lại, các lệnh sẽ được thực thi bình thường. Cơ chế này giúp ngăn chặn việc cập nhật dữ liệu không nhất quán, tuy nhiên không cung cấp khả năng rollback như trong MySQL.

Để minh họa cơ chế phát hiện xung đột này, đoạn mã dưới đây mô phỏng hai client truy cập đồng thời vào cùng một key:

```
1 -- Terminal 1
2 redis-cli WATCH production:stock:1:1
3 redis-cli MULTI
4 redis-cli HINCRBY production:stock:1:1 quantity -1
5
6 -- Terminal 2:
7 redis-cli HSET production:stock:1:1 quantity 100
8
9 -- Terminal 1
10 redis-cli EXEC
```

```
127.0.0.1:6379(TX)> EXEC  
(nil)  
127.0.0.1:6379> █
```

Hình 2.53: Kết quả sau khi thực thi các câu lệnh trên

2.4.3 Nhận xét so sánh

- MySQL đảm bảo transaction ở mức hệ quản trị cơ sở dữ liệu, với đầy đủ các thuộc tính ACID, đặc biệt là khả năng rollback nhằm khôi phục trạng thái dữ liệu khi xảy ra lỗi.
- Redis chỉ đảm bảo tính nguyên tử (atomicity) ở mức thực thi lệnh hoặc khối lệnh (thông qua MULTI/EXEC hoặc Lua script), nhưng không hỗ trợ cơ chế rollback khi xảy ra sai sót logic.
- Trong Redis, trách nhiệm đảm bảo tính đúng đắn và nhất quán của dữ liệu được chuyển sang tầng ứng dụng hoặc logic xử lý (ví dụ: kiểm tra điều kiện trước khi cập nhật bằng Lua script).

Nhìn chung, MySQL phù hợp với các hệ thống yêu cầu tính toàn vẹn dữ liệu nghiêm ngặt và cơ chế kiểm soát transaction mạnh mẽ, trong khi Redis ưu tiên hiệu năng và tính đơn giản, chấp nhận đánh đổi bằng việc giảm bớt các cơ chế bảo vệ dữ liệu ở mức hệ quản trị.

2.5 Concurrency

2.5.1 Khái niệm Concurrency Control

Concurrency Control là tập hợp các kỹ thuật để đảm bảo tính đúng đắn của dữ liệu khi nhiều client truy cập và thao tác trên cùng dữ liệu cùng một lúc. Một số lỗi thường gặp bao gồm:

- **Lost Update:** 2 transactions cùng ghi khiến 1 update bị mất.
- **Dirty Read:** Đọc dữ liệu chưa commit từ transaction khác.
- **Non-repeatable Read:** Đọc 2 lần ra 2 kết quả khác nhau.

2.5.2 Kiến trúc 2 DBMS

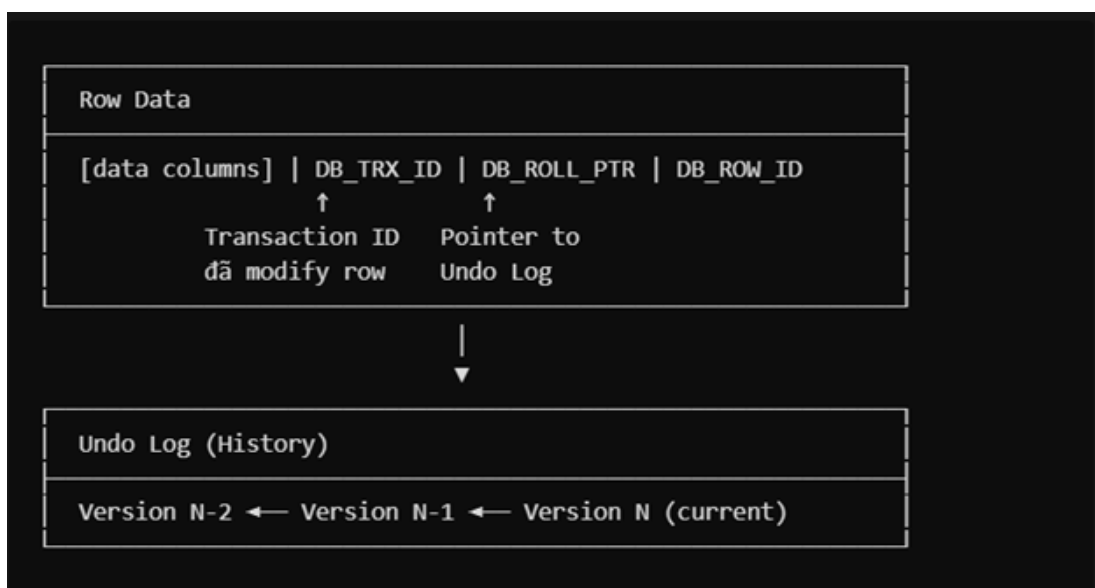
Dưới đây là bảng so sánh kiến trúc giữa MySQL và Redis:

Bảng 2.7: So sánh kiến trúc MySQL và Redis

Tiêu chí	MySQL	Redis
Mô hình	Multi-threaded	Single-threaded
Cơ chế	MVCC & Locking	Single-threaded Execution
Mục tiêu	Đảm bảo ACID	Đảm bảo Performance

MySQL Mỗi connection được gán với 1 thread riêng, có thể đọc / ghi đồng thời trên cùng dữ liệu. Do đó, MySQL cần các cơ chế lock để kiểm soát truy cập:

- **Shared Lock:** Cho phép nhiều transaction cùng đọc.
- **Exclusive Lock:** Cho phép đúng 1 transaction được đọc / ghi.
- **MVCC (Multi-Version Concurrency Control):** Cho phép readers không block writers và ngược lại bằng cách duy trì nhiều phiên bản của dữ liệu. Khi tạo transaction, nó sẽ lưu trữ một Read View và chỉ có thể đọc các data ở phiên bản phù hợp.



Hình 2.54: Minh họa cơ chế MVCC

Redis Redis hoạt động Single-threaded, mọi lệnh của Redis (INCR, HINCRBY, v.v.), các transaction (MULTI / EXEC), Lua Script đều nguyên tử và được thực thi tuần tự. Điều này đảm bảo rằng không có 2 lệnh nào chạy đồng thời, do đó các transaction được xử lý tuần tự, không gặp race-condition, không phải dùng các cơ chế lock phức tạp.

2.5.3 Thực nghiệm

Môi trường kiểm thử Các thông số môi trường thử nghiệm:

Bảng 2.8: Thông số môi trường kiểm thử

Thông số	Giá trị
Cores	8
Base Clock	3.1 GHz
OS	Window 11
MySQL version	8.0.x (InnoDB)
Redis version	Memurai (Window)
Dataset	Sales Transaction – 536k transactions
Python	3.13 + mysql-connector-python, redis

Test đọc đồng thời Đo throughput khi nhiều client (thread) đọc dữ liệu ngẫu nhiên đồng thời.

Mã nguồn Python minh họa:

```
def mysql_worker_pooled(thread_id):
    conn = mysql_pool.get_connection()
    cursor = conn.cursor()
    start = time.time()
    for i in range(NUM_READS):
        tid = random.randint(1, TOTAL_TRANSACTIONS)
        cursor.execute("SELECT * FROM transactions WHERE id = %s", (tid))
        cursor.fetchall()
    elapsed = time.time() - start
    conn.close()
    return elapsed

def redis_worker_pooled(thread_id):
    r = redis.Redis(connection_pool=redis_pool)
    start = time.time()
    for i in range(NUM_READS):
        tid = random.randint(1, TOTAL_TRANSACTIONS)
        r.hgetall(f"transaction:{tid}")
    elapsed = time.time() - start
    return elapsed
```

```
=====
TEST 5: Scale Test (Threads vs Throughput)
=====
Reads per thread: 500
```

Threads	MySQL (ops/s)	Redis (ops/s)
1	5,557	13,809
2	9,897	14,807
3	12,789	13,968
4	15,087	13,376
5	14,731	13,556
10	12,270	12,575
20	12,167	13,178
30	12,156	13,162

Hình 2.55: Kết quả test đọc đồng thời

Nhận xét:

- MySQL tăng trưởng theo số luồng tốt vì tận dụng multi-threading tốt.
- Đến tầm 10 threads, hiệu năng MySQL bắt đầu đi ngang và giảm nhẹ do context switching và tác dụng ngược của MVCC.
- Redis đạt đỉnh từ 2–5 threads và không thể tăng thêm do đơn luồng.
- Khi ít thread, Redis nhanh hơn, nhưng khi nhiều thread, MySQL có xu hướng vượt trội hơn.

Test ghi đồng thời Đo throughput khi nhiều client ghi dữ liệu đồng thời.

```
=====
TEST 4: Concurrent Write Scale Test
=====
Writes per thread: 50
```

Threads	MySQL (ops/s)	Redis (ops/s)	Winner
1	353	9,742	Redis
2	323	11,766	Redis
3	407	11,001	Redis
4	348	10,536	Redis
5	391	10,354	Redis
10	391	10,918	Redis
20	388	10,742	Redis
30	390	10,566	Redis

Hình 2.56: Kết quả test ghi đồng thời

Nhận xét:

- Tốc độ ghi của MySQL chậm hơn Redis nhiều lần bất kể số lượng thread.
- **Nguyên nhân:** MySQL cần duy trì tính ACID, ghi xuống đĩa và lock dữ liệu gây bottleneck. Redis hoạt động trên RAM và không tồn tại tranh chấp khóa.

```
=====
TEST 2: Varying Read/Write Ratio (3 threads, 500 ops/thread)
=====
```

Ratio	MySQL tput	Redis tput	MySQL p95	Redis p95	Winner
Fully Read (100/0)	12,539	13,178	0.36ms	0.39ms	Redis
Read-heavy (90/10)	3,388	13,266	5.54ms	0.38ms	Redis
Balanced (50/50)	782	12,458	8.40ms	0.43ms	Redis
Write-heavy (10/90)	436	11,932	8.80ms	0.44ms	Redis
Fully Write (0/100)	399	12,341	8.56ms	0.43ms	Redis

```
○ (.venv) PS C:\Users\khe1a\Downloads\DBMS> █
```

Hình 2.57: Kết quả test đọc ghi hỗn hợp theo tỉ lệ

Test đọc ghi hỗn hợp Nhận xét:

- MySQL vẫn cho kết quả tốt khi lượng đọc vẫn còn ít, càng đọc nhiều thì throughput càng giảm.
- Redis cho throughput tốt và ổn định dù ở tỉ lệ nào.

2.5.4 Cơ chế lock trong MySQL

MySQL sử dụng Shared Lock (S-Lock) và Exclusive Lock (X-Lock):

- **Shared Lock:** Các transaction khác vẫn có thể đọc nhưng phải chờ nếu muốn ghi.
- **Exclusive Lock:** Không transaction nào khác được đọc hay ghi vào hàng đó.

```
=====
TEST 1: Multiple Shared Locks (FOR SHARE) - Compatible
=====
```

Scenario:

```
T1: SELECT ... FOR SHARE (acquire S-Lock)
T2: SELECT ... FOR SHARE (acquire S-Lock)
T3: SELECT ... FOR SHARE (acquire S-Lock)
```

Expected: All 3 get lock immediately (S-Locks are compatible)

Execution:

```
-----
[16:54:03.376] T3: Got S-Lock in 0.6ms, value = 100
[16:54:03.377] T1: Got S-Lock in 0.5ms, value = 100
[16:54:03.377] T2: Got S-Lock in 0.5ms, value = 100
[16:54:03.879] T1: Released S-Lock
[16:54:03.879] T3: Released S-Lock
[16:54:03.879] T2: Released S-Lock
-----
```

Results:

```
T1 lock time: 0.5ms
T2 lock time: 0.5ms
T3 lock time: 0.6ms
```

Hình 2.58: Test lock – Shared Lock cho phép đọc đồng thời

Scenario:

```
T1: SELECT ... FOR SHARE (holds S-Lock for 2 seconds)
T2: SELECT ... FOR UPDATE (waits for X-Lock)
```

Expected: T2 must wait until T1 releases S-Lock

Execution:

```
-----
[16:54:03.934] T1 (S): Acquired S-Lock, value = 100, holding for 2s...
[16:54:04.045] T2 (X): Requesting X-Lock (FOR UPDATE)...
[16:54:05.937] T2 (X): Got X-Lock after 1.89s, value = 100
[16:54:05.937] T1 (S): Released S-Lock
-----
```

Results:

```
T2 waited: 1.89s for X-Lock
```

```
✓ X-Lock request was BLOCKED by existing S-Lock
✓ T2 got lock only after T1 committed
```

Hình 2.59: Test lock – Shared Lock chặn ghi

```
=====
TEST 3: Exclusive Lock Blocks Shared Lock
=====
```

Scenario:

```
T1: SELECT ... FOR UPDATE (holds X-Lock for 2 seconds)
T2: SELECT ... FOR SHARE (waits for S-Lock)
```

Expected: T2 must wait until T1 releases X-Lock

Execution:

```
-----
[19:12:00.465] T1 (X): Acquired X-Lock, value = 100, holding for 2s...
[19:12:00.572] T2 (S): Requesting S-Lock (FOR SHARE)...
[19:12:02.466] T1 (X): Released X-Lock
[19:12:02.466] T2 (S): Got S-Lock after 1.89s, value = 100
-----
```

Results:

T2 waited: 1.89s for S-Lock

- ✓ S-Lock request was BLOCKED by existing X-Lock
- ✓ T2 got lock only after T1 committed

Hình 2.60: Test lock – Exclusive Lock chặn đọc và ghi

```
=====
TEST 4: Exclusive Lock Blocks Exclusive Lock
=====
```

```
Scenario:
```

```
T1: SELECT ... FOR UPDATE (holds X-Lock for 2 seconds)
```

```
T2: SELECT ... FOR UPDATE (waits for X-Lock)
```

```
Expected: T2 must wait until T1 releases X-Lock
```

```
Execution:
```

```
-----
[19:12:02.482] T1: Got X-Lock in 0.00s, value = 100, holding for 2.0s...
[19:12:02.589] T2: Requesting X-Lock...
[19:12:04.483] T1: Released X-Lock
[19:12:04.486] T2: Got X-Lock in 1.90s, value = 100
[19:12:04.487] T2: Released X-Lock
-----
```

```
Results:
```

```
T1 got lock immediately: 0.4ms
```

```
T2 waited: 1.90s
```

```
✓ Only ONE X-Lock per row at a time
```

```
✓ X-Lock is EXCLUSIVE - blocks all other locks
```

Hình 2.61: Test lock – Tổng hợp kết quả

2.5.5 Cơ chế MVCC của MySQL

```
=====
TEST 1: REPEATABLE READ - Consistent Snapshot
=====

MVCC Concept:
- Khi transaction bắt đầu, MySQL tạo "snapshot" của database
- Transaction chỉ thấy data từ snapshot đó
- Writers tạo VERSION MỚI, không ghi đè version cũ
- Readers cũ vẫn đọc được version cũ

Initial: balance = 1000

Timeline:
-----
[13:04:26.807] Reader: BEGIN (REPEATABLE READ) - Snapshot created!
[13:04:26.808] Reader: SELECT balance = 1000
[13:04:27.017] Writer: BEGIN
[13:04:27.019] Writer: UPDATE balance = 500
[13:04:27.051] Writer: COMMIT - New version created!
[13:04:27.152] Reader: SELECT balance = 1000 (SAME as before!)
[13:04:27.655] Reader: SELECT balance = 1000 (Still consistent!)
[13:04:27.656] Reader: COMMIT
-----

Results:
Reader's observations: [1000, 1000, 1000]
Writer committed:      500
Actual in DB:          500
```

Hình 2.62: Tính Repeatable Read của MySQL nhờ MVCC

Nhận xét:

- **Điểm tạo Snapshot:** Ngay khi Reader thực hiện lệnh SELECT đầu tiên (hoặc BEGIN), một "bản chụp" dữ liệu tại thời điểm đó được xác lập.
- **Độc lập với Writers:** Sau khi snapshot được tạo, Reader sẽ chỉ nhìn thấy dữ liệu tại thời điểm đó, bất kể Writer có thực hiện UPDATE/INSERT/DELETE nào sau đó. Điều này đảm bảo tính Repeatable Read.

```
=====
TEST 4: Multiple Readers - Different Snapshots
=====

Scenario:
  Initial balance = 1000

Timeline:
  R1 starts (snapshot = 1000)
  W1 updates to 800
  R2 starts (snapshot = 800)
  W2 updates to 600
  R3 starts (snapshot = 600)

All readers final read...

-----
[13:04:30.310] Reader1: BEGIN, snapshot balance = 1000
[13:04:30.455] Writer1: UPDATE balance = 800, COMMIT
[13:04:30.567] Reader2: BEGIN, snapshot balance = 800 (after W1)
[13:04:30.711] Writer2: UPDATE balance = 600, COMMIT
[13:04:30.914] Reader1: Final read = 1000 (snapshot from start)
[13:04:30.914] Reader2: Final read = 800
[13:04:30.921] Reader3: BEGIN, snapshot balance = 600 (after W2)
-----

Results:
R1 (started before W1): saw [1000, 1000] → Snapshot = 1000
R2 (started after W1): saw [800, 800] → Snapshot = 800
R3 (started after W2): saw [600] → Snapshot = 600
```

Hình 2.63: Quản lý đa phiên bản (Multi-versioning) trong MVCC

Nhận xét:

- **Đa phiên bản:** MySQL không chỉ giữ một phiên bản mà có thể quản lý nhiều phiên bản cùng lúc
- **Phiên bản dữ liệu:** Mỗi khi Writer thực hiện UPDATE, một phiên bản mới của hàng được tạo ra. Reader sẽ chỉ nhìn thấy phiên bản tại thời điểm snapshot được tạo, không bị ảnh hưởng bởi các phiên bản mới.

Ưu điểm về hiệu năng: Reader không bị block bởi Writer và ngược lại, giúp tăng hiệu năng khi có nhiều transaction đọc và ghi đồng thời.

2.5.6 Deadlock trong MySQL

MySQL có cơ chế kiểm tra deadlock; nếu phát hiện vòng lặp chờ đợi (circular wait), nó sẽ rollback một transaction để giải tỏa nghẽn. Redis không xảy ra deadlock do kiến trúc single-threaded.

```
=====
TEST 3: Deadlock Detection (MySQL Only)
=====

Scenario:
  Thread 1: Lock A → wait → Lock B
  Thread 2: Lock B → wait → Lock A
  Expected: DEADLOCK (circular wait)

Results (after 0.52s):
-----
t1: deadlock
    Error: 1213 (40001): Deadlock found when trying to get lock; try restarting transaction...
t2: success

✓ DEADLOCK DETECTED - MySQL rolled back one transaction

[Redis Note]
  Redis: KHÔNG thể xảy ra deadlock (single-threaded)
```

Hình 2.64: Kết quả test Deadlock

2.5.7 Tính nguyên tử của Redis

Redis chạy đơn luồng nên các lệnh đơn nguyên hoặc Lua script sẽ được xử lý tuần tự, đảm bảo tính nguyên tử (atomic) mà không cần lock.

```
=====
TEST: Redis Lua Script Atomicity
=====

Scenario: 10 threads try to decrement stock by 15 each
Initial stock: 100, Total requested: 150
Expected: Only 6 decrements succeed (100/15=6), final stock: 10

Results:
  Successful decrements: 6
  Failed decrements: 4
  Final stock: 10
  ✓ ATOMIC: Final stock matches expected calculation
  No race conditions - Lua script executed atomically
```

Hình 2.65: Minh họa tính nguyên tử với Lua script trong Redis

2.5.8 Kết luận

Bảng tổng kết ưu và nhược điểm của hai hệ thống:

Bảng 2.9: So sánh ưu nhược điểm MySQL và Redis về concurrency

	MySQL (Multi-thread)	Redis (Single-thread)
Ưu điểm	Scale với CPU cores, chạy song song query, dùng MVCC hiệu quả.	Đơn giản, tốc độ cao, không lock overhead, latency ổn định.
Nhược điểm	Phức tạp, cần quản lý lock và tune isolation level.	Bottleneck ở 1 thread, không tận dụng được đa nhân, tác vụ dài gây nghẽn.



Reference Link

1. Reference Link to Github: <https://github.com/KhaLe1412/SoftwareEngineering.git>
2. Reference Link to Figma: <https://www.figma.com/make/00kQiiqXgiGapfIcdJLwvS/Tutor-Student-Management-System?t=Ci9fSiMSlLRKqwql-20&fullscreen=1>